The Java™ Tutorials

Trail: Learning the Java Language

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Lesson: Object-Oriented Programming Concepts

If you've never used an object-oriented programming language before, you'll need to learn a few basic concepts before you can begin writing any code. This lesson will introduce you to objects, classes, inheritance, interfaces, and packages. Each discussion focuses on how these concepts relate to the real world, while simultaneously providing an introduction to the syntax of the Java programming language.

What Is an Object?

An object is a software bundle of related state and behavior. Software objects are often used to model the real-world objects that you find in everyday life. This lesson explains how state and behavior are represented within an object, introduces the concept of data encapsulation, and explains the benefits of designing your software in this manner.

What Is a Class?

A class is a blueprint or prototype from which objects are created. This section defines a class that models the state and behavior of a real-world object. It intentionally focuses on the basics, showing how even a simple class can cleanly model state and behavior.

What Is Inheritance?

Inheritance provides a powerful and natural mechanism for organizing and structuring your software. This section explains how classes inherit state and behavior from their superclasses, and explains how to derive one class from another using the simple syntax provided by the Java programming language.

What Is an Interface?

An interface is a contract between a class and the outside world. When a class implements an interface, it promises to provide the behavior published by that interface. This section defines a simple interface and explains the necessary changes for any class that implements it.

What Is a Package?

A package is a namespace for organizing classes and interfaces in a logical manner. Placing your code into packages makes large software projects easier to manage. This section explains why this is useful, and introduces you to the Application Programming Interface (API) provided by the Java platform.

Questions and Exercises: Object-Oriented Programming Concepts

Use the questions and exercises presented in this section to test your understanding of objects, classes, inheritance, interfaces, and packages.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Table of Contents
Next page: What Is an Object?

| Ad Choices

The Java™ Tutorials

Trail: Learning the Java Language

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Lesson: Classes and Objects

With the knowledge you now have of the basics of the Java programming language, you can learn to write your own classes. In this lesson, you will find information about defining your own classes, including declaring member variables, methods, and constructors.

You will learn to use your classes to create objects, and how to use the objects you create.

This lesson also covers nesting classes within other classes, and enumerations

Classes

This section shows you the anatomy of a class, and how to declare fields, methods, and constructors.

Objects

This section covers creating and using objects. You will learn how to instantiate an object, and, once instantiated, how to use the `dot` operator to access the object's instance variables and methods.

More on Classes

This section covers more aspects of classes that depend on using object references and the `dot` operator that you learned about in the preceding section: returning values from methods, the `this` keyword, class vs. instance members, and access control.

Nested Classes

Static nested classes, inner classes, anonymous inner classes, local classes, and lambda expressions are covered. There is also a discussion on when to use which approach.

Enum Types

This section covers enumerations, specialized classes that allow you to define and use sets of constants.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Previous Lesson
Next page: Classes

| Ad Choices

The Java™ Tutorials

Trail: Learning the Java Language
Lesson: Interfaces and Inheritance

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Inheritance

In the preceding lessons, you have seen *inheritance* mentioned several times. In the Java language, classes can be *derived* from other classes, thereby *inheriting* fields and methods from those classes.

Definitions: A class that is derived from another class is called a *subclass* (also a *derived class*, *extended class*, or *child class*). The class from which the subclass is derived is called a *superclass* (also a *base class* or a *parent class*).

Excepting `Object`, which has no superclass, every class has one and only one direct superclass (single inheritance). In the absence of any other explicit superclass, every class is implicitly a subclass of `Object`.

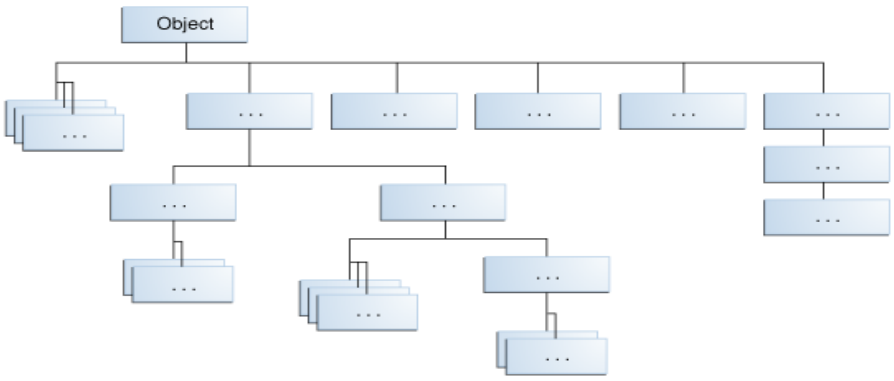
Classes can be derived from classes that are derived from classes that are derived from classes, and so on, and ultimately derived from the topmost class, `Object`. Such a class is said to be *descended* from all the classes in the inheritance chain stretching back to `Object`.

The idea of inheritance is simple but powerful: When you want to create a new class and there is already a class that includes some of the code that you want, you can derive your new class from the existing class. In doing this, you can reuse the fields and methods of the existing class without having to write (and debug!) them yourself.

A subclass inherits all the *members* (fields, methods, and nested classes) from its superclass. Constructors are not members, so they are not inherited by subclasses, but the constructor of the superclass can be invoked from the subclass.

The Java Platform Class Hierarchy

The `Object` class, defined in the `java.lang` package, defines and implements behavior common to all classes—including the ones that you write. In the Java platform, many classes derive directly from `Object`, other classes derive from some of those classes, and so on, forming a hierarchy of classes.



All Classes in the Java Platform are Descendants of Object

At the top of the hierarchy, `Object` is the most general of all classes. Classes near the bottom of the hierarchy provide more specialized behavior.

| Ad Choices

An Example of Inheritance

Here is the sample code for a possible implementation of a `Bicycle` class that was presented in the Classes and Objects lesson:

```
public class Bicycle {

    // the Bicycle class has three fields
    public int cadence;
    public int gear;
    public int speed;

    // the Bicycle class has one constructor
    public Bicycle(int startCadence, int startSpeed, int startGear) {
        gear = startGear;
        cadence = startCadence;
        speed = startSpeed;
    }

    // the Bicycle class has four methods
    public void setCadence(int newValue) {
        cadence = newValue;
    }

    public void setGear(int newValue) {
        gear = newValue;
    }

    public void applyBrake(int decrement) {
        speed -= decrement;
    }

    public void speedUp(int increment) {
        speed += increment;
    }

}
```

A class declaration for a `MountainBike` class that is a subclass of `Bicycle` might look like this:

```
public class MountainBike extends Bicycle {

    // the MountainBike subclass adds one field
    public int seatHeight;

    // the MountainBike subclass has one constructor
    public MountainBike(int startHeight,
                        int startCadence,
                        int startSpeed,
                        int startGear) {
        super(startCadence, startSpeed, startGear);
        seatHeight = startHeight;
    }

    // the MountainBike subclass adds one method
    public void setHeight(int newValue) {
        seatHeight = newValue;
    }
}
```

`MountainBike` inherits all the fields and methods of `Bicycle` and adds the field `seatHeight` and a method to set it. Except for the constructor, it is as if you had written a new `MountainBike` class entirely from scratch, with four fields and five methods. However, you didn't have to do all the work. This would be especially valuable if the methods in the `Bicycle` class were complex and had taken substantial time to debug.

What You Can Do in a Subclass

A subclass inherits all of the *public* and *protected* members of its parent, no matter what package the subclass is in. If the subclass is in the same

| Ad Choices

or supplement them with new members:

- The inherited fields can be used directly, just like any other fields.
- You can declare a field in the subclass with the same name as the one in the superclass, thus *hiding* it (not recommended).
- You can declare new fields in the subclass that are not in the superclass.
- The inherited methods can be used directly as they are.
- You can write a new *instance* method in the subclass that has the same signature as the one in the superclass, thus *overriding* it.
- You can write a new *static* method in the subclass that has the same signature as the one in the superclass, thus *hiding* it.
- You can declare new methods in the subclass that are not in the superclass.
- You can write a subclass constructor that invokes the constructor of the superclass, either implicitly or by using the keyword `super`.

The following sections in this lesson will expand on these topics.

Private Members in a Superclass

A subclass does not inherit the `private` members of its parent class. However, if the superclass has public or protected methods for accessing its private fields, these can also be used by the subclass.

A nested class has access to all the private members of its enclosing class—both fields and methods. Therefore, a public or protected nested class inherited by a subclass has indirect access to all of the private members of the superclass.

Casting Objects

We have seen that an object is of the data type of the class from which it was instantiated. For example, if we write

```
public MountainBike myBike = new MountainBike();
```

then `myBike` is of type `MountainBike`.

`MountainBike` is descended from `Bicycle` and `Object`. Therefore, a `MountainBike` is a `Bicycle` and is also an `Object`, and it can be used wherever `Bicycle` or `Object` objects are called for.

The reverse is not necessarily true: a `Bicycle` *may be* a `MountainBike`, but it isn't necessarily. Similarly, an `Object` *may be* a `Bicycle` or a `MountainBike`, but it isn't necessarily.

Casting shows the use of an object of one type in place of another type, among the objects permitted by inheritance and implementations. For example, if we write

```
Object obj = new MountainBike();
```

then `obj` is both an `Object` and a `MountainBike` (until such time as `obj` is assigned another object that is *not* a `MountainBike`). This is called *implicit casting*.

If, on the other hand, we write

```
MountainBike myBike = obj;
```

we would get a compile-time error because `obj` is not known to the compiler to be a `MountainBike`. However, we can *tell* the compiler that we promise to assign a `MountainBike` to `obj` by *explicit casting:*

```
MountainBike myBike = (MountainBike)obj;
```

This cast inserts a runtime check that `obj` is assigned a `MountainBike` so that the compiler can safely assume that `obj` is a `MountainBike`. If `obj` is not a `MountainBike` at runtime, an exception will be thrown.

Note: You can make a logical test as to the type of a particular object using the `instanceof` operator. This can save you from a runtime error owing to an improper cast. For example:

```
if (obj instanceof MountainBike) {
    MountainBike myBike = (MountainBike)obj;
}
```

Here the `instanceof` operator verifies that `obj` refers to a `MountainBike` so that we can make the cast with knowledge that there will be no runtime exception thrown.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

| Ad Choices

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Questions and Exercises: Interfaces

Next page: Multiple Inheritance of State, Implementation, and Type

| Ad Choices

Module 4: Classification and Composition

Topics

1. Coupling and Cohesion (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m4-module-4/s3-commentary.html?ou=693191#I)
2. Choosing Composition or Classification (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m4-module-4/s3-commentary.html?ou=693191#II)
3. Aggregation and Acquaintance (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m4-module-4/s3-commentary.html?ou=693191#III)

I. Coupling and Cohesion

One way to evaluate the design of a method, object, or even complete system is to consider its use of coupling and cohesion. Coupling and cohesion affect the structural integrity of a program, and thus impact the overall extensibility, stability, and maintenance of the program. These properties are hard to quantify, though some systems that measure program metrics, such as *Structural Analysis for Java* (http://www.alphaworks.ibm.com/tech/sa4j) , do attempt to quantify them. More important is their usefulness in helping you to rate the quality of two design options. This module will use coupling and cohesion to evaluate overall design options such as when to use classification and composition, and how to use aggregation or acquaintance relationships.

In a larger sense, simply knowing the coupling and cohesion characteristics of a program as it is being built will help a programmer understand the behavior of a program. It will also help programmers find programming problems and make better

decisions as they implement a program. It is with these issues in mind that we will cover coupling and cohesion.

A. Definition of Coupling and Cohesion

Coupling is the degree to which one object or method needs to know, or is given the opportunity to know, something about another object or method. For example, when one object can affect the internal details of another object, those two objects are considered to be tightly coupled. Coupling can occur in many ways in Java. It can come about through the use of a reference or through an extension of an object. The most obvious example of a tightly coupled system is one in which the variables from one object are made public and manipulated by another object. This is shown in the following Person class:

```
public class Person {
String _name;
int _age;

public Person (String name, int age) {
_name = name;
if (age > 0 && age < 120)
_age = age;
else
_age = 0;
}

public void setAge(int age) {
if (age > 0 && age < 120)
_age = age;
else
_age = 0;
}
}

class UseAge {
```

```
public static void main(String[] args) {
Person p = new Person("John Doe", 25);
p._age = -15;
}
}
```

In the above program, the programmer has not made the variable _age private, so any program can access this variable. This _age variable is used in the UseAge class's main() method to unsafely set the _age to an invalid value of -15, which violates a class invariant. UseAge is coupled to the Person class, in that changes to the Person class need to be accounted for in the UseAge class. This is an extreme example of high coupling, but one that does exist in many programs.

Cohesion is the degree to which a class or method has simple, well-defined limits on its purpose and/or functionality. A lack of cohesion is like a lack of focus. The class shown below is a simplified version of one presented at the Microsoft 2005 Academic Discussion to illustrate an instance in which poor programming had made it into a production system. This example highlights a method that lacks cohesion. A single setValue() method is implemented in a Person class. This method uses a flag value to determine what variable to set in this method. Note that even though the _age variable in this method is an int—so that the method will work with any data—the value will be passed in as a String. This system would be much more robust if separate methods were written for each variable in the Person class, and that the set methods for the variables were required to take parameters of the correct class.

```
public class Person {
public enum VariableType {AGE, NAME};

String _name;
int _age;

public void setVariable(VariableType type, String value) {
if (type == VariableType.AGE) {
int iValue = Integer.parseInt(value);
```

```
if (iValue > 0 && iValue < 125)
_age = iValue;
else
_age = 0;
}
else if (type == VariableType.NAME) {
_name = value;
}
}
}
```

It is important to remember that coupling and cohesion are factors that come into play when designing both classes and methods. Although this module will generally deal with problems in classes, it is important to remember that these problems can be present in classes and methods, and that the design of each comes with its own unique problems.

In general, the more loosely coupled and tightly cohesive a design is, the better the design. Why? First, as coupling increases, the complexity of the design increases because of the possible side effects changes in one object could cause in another object. Second, a less cohesive object or method will be less well defined in what it can and should do. An object or method that does a number of things normally does none of them as well as something that has a single focus.

Poor decisions about coupling and cohesion are common in real life. For example, it would not be uncommon for a programmer to create a method for a Person object in which the Person object is read from a database. A programmer may then decide the method would be much more powerful if the user could actually manipulate the underlying data. A method could then be designed that takes a String that is the SQL select statement for the fields to be selected from a Person database table. In this case, what is returned from the method is an object array of the items the programmer has selected. This process is illustrated in the code fragment below:

```
public Object[] getPersonData(String s) ...
...
Object[] o = getPersonData("Select name, age, sex");
```

This appears to be a very powerful and useful method. Rather than forcing the programmer to write methods for each data field available in the table, this method allows the user to select just those fields in which they are interested. The problem with this type of method is that it is both too highly coupled to the internal representation of the object (the database used and the data fields in that particular database), and too loosely cohesive, as the method can be used for any query against the database. For example, you can tell that this method is too highly coupled because the query requires that the programmer using the getPersonData() method know a lot about the Person object—how the data is stored, the names of the data fields, etc. The programmer maintaining the Person object cannot change these details without affecting the programs that use this object.

This method is also too loosely cohesive in that it allows the programmer to do too many things with the object. For example, a programmer will probably realize that the object takes her select string, and appends to it to get the following query:

Select name, age, sex from Person

Given this behavior, there is no protection from an accidental (or even malicious) use of this method as follows:

Object[] o = getPersonData("Delete *");

This use of the method would delete all records in the table, which is not likely the intention of the author of the Person object. Making an object or method more loosely coupled and more tightly cohesive is almost always an advantage.

This rule does not imply that a highly cohesive or loosely coupled method or class is not complex. For example, consider the method "boolean approveMortgage(String customer, float amount)". This method would probably implement a number of interactions (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m4-module-4/popups/popup4-i-1.html?ou=693191) with other objects, as well as methods to determine the credit score of the person applying for the loan, to appraise the property, and so on. These interactions could even represent remote

network connects. The entire implementation of this method could be very complex. However, the program calling this method is completely unaware of the details or the complexity of the underlying transaction, so the method is loosely coupled. Also the program calling this method sees only a simple transaction of approving the mortgage, and thus, the method is highly cohesive.

Decreasing coupling and increasing cohesion in a program is often a worthy goal to aspire to when writing a program. This process can be summed up as keep things separate and simple, and the more separate and simple, the better.

II. Choosing Composition or Classification

This section will explore the decision to use composition or classification in a design. It will not discuss the design of objects, but instead deal with program errors that can occur when classification is chosen rather than composition. The final result—that you should always favor composition over classification—will be justified by showing that classification tends to create rigid and fragile designs. However, classification does have its uses, and we provide rules for when to properly use classification later in this module.

A. Problems with Classification

The easy answer to whether to use classification or composition is to simply state that, in Java, combing classes using classification (or inheritance) creates one object, a single entity that represents all properties of that object. Building an object by combining objects from multiple classes (composition) would create a design where the parts of the object are physically separate entities. Designs with single entities will generally be more tightly coupled and loosely cohesive than ones that make use of multiple entities. As a general rule, designs that use composition are safer than designs that use classification. But are designs that use composition more loosely coupled and tightly cohesive than designs that use classification? If so, which types of problems are avoided by using composition rather than classification? If these issues are not understood, programmers will not know how to use the rule, and perhaps more importantly, will not know how to break this rule effectively and safely.

As with any rule, the rule regarding composition and classification exists because it is useful. Understanding why a rule is useful, and why it was invented, is always more useful than trying to remember all the situations in which the rule applies and all the possible exceptions to the rule. Once you know why something works, you can more easily decide how to apply it. This section will show you how the coupling that occurs in classification leads to problems. This will help you avoid these problems and, just as importantly, to understand when classification represents a proper solution to a problem.

Do Not Use Classification if the Type of an Object Can Change

The first rule we will give is that you should not use classification if the type of an object can change. This can be more simply stated by saying that, since the general rule is that classification is called an *is-a* relationship, this rule can be called an is-always-a extension to the is-a rule.

Violation of the classification rule can result in extremely harmful program bugs. These bugs may be hard or impossible to fix, and may have very far-reaching and significant consequences. Worse, these bugs are hard to avoid, and often only show up after a system is in production and data for the system has been compromised, often to the point where the data is irrecoverable. By the end of this section, you will see why the classification rule is a very important rule to always consider before implementing objects that can change type, and why you should avoid violating the rule if at all possible.

To understand the classification rule, consider the following test case that we can add to our unit testing of the Bank application. In this example, the customer changes from a BlueCustomer to a SilverCustomer. This change is then made in the bank so that the Customer John Doe is now a SilverCustomer.

```
import bank.*;
import java.lang.reflect.*;
import java.util.GregorianCalendar;
import java.util.Collection;

public class TestBank {
public static void main(String[] args) {
```

```
Bank bank = null;
try {
bank = Bank.getBank();

//First create the customer, then add the account;
bank.addCustomer(new BlueCustomer(
new Name("John", "Doe"),
new GregorianCalendar(1985, 3, 13)));
Account acct = new CheckingAccount(
new Name("John", "Doe"), 2800.00F);
bank.addAccount(acct);

// Now change John Doe from Blue to Silver Customer
Customer cust = bank.getCustomer(
new Name("John", "Doe"));
cust = new SilverCustomer(cust, 1000.00f);
bank.changeCustomerType(cust);

// Print out John Doe's type in the Bank
cust = bank.getCustomer(new Name("John", "Doe"));
System.out.println("Customer = John Doe is a " +
cust.getClass() + " customer");

// Print out John Doe's type in the account
cust = bank.getAccount(
acct.getAccountNumber()).getAccountOwner();
System.out.println("Customer = John Doe is a " +
cust.getClass() + " customer");

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

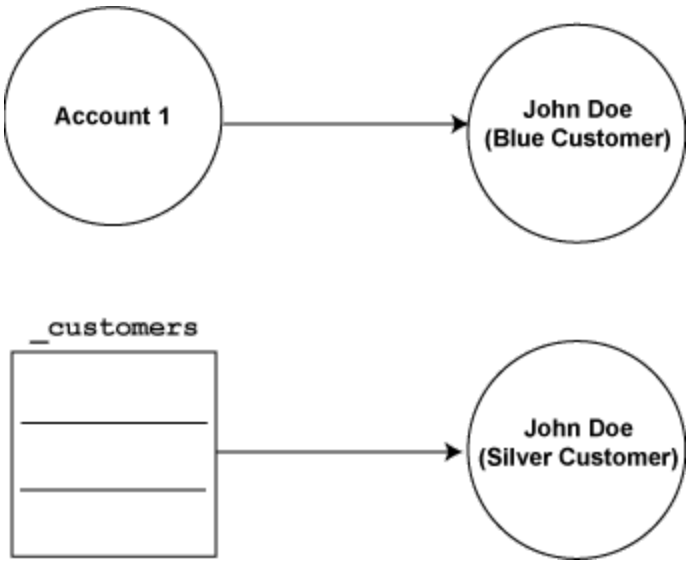
When we run this test case, we get the following result:

Customer = John Doe is a class bank.SilverCustomer customer
Customer = John Doe is a class bank.BlueCustomer customer

Now John Doe is both a BlueCustomer and a SilverCustomer in our bank. How is this possible? It is because, when the customer type for John Doe was changed, the object was forced to change types. When an object changes types, the current object is not simply changed to reflect the new data—a new object is created. This means that every reference to that object everywhere it is used in the system must be updated. The bank program did not update all references to the John Doe Customer object, and so two objects remain in the program for John Doe.

We can show this graphically in figure 4.1. In this figure, the Customer is referenced both from the _customers Map in the bank, but also from the individual accounts. When we changed the type, the reference was changed in the _customers Map, but it was not changed in the individual accounts. Thus, existing accounts will all now reference the old Customer object, and the _customers Map and any new accounts will reference the new Customer object.

Figure 4.1



References to the Customer Object in Classification Design of the Bank

This necessity of an object to change types is a problem in designs that use classification and is often not acknowledged by designers. Many designers feel that the design should represent what they, as the designer, see as the relationships in the systems. Design is often seen as being completely unrelated to the implementation in a programming language.

To many designers, the problem is that the programmer simply did not take into account all the side effects of changing a customer type. After all, no bug existed until the programmer tried to change the type, so the code that changed the type is the direct cause of the bug. Designers might feel that it is the responsibility of the programmer to account for all side effects in every change they make, regardless of the problems exposed in the design.

This expectation would be fine if it were possible to easily predict and account for all the side effects that could occur in changing a type, and if the designers took all these side effects into consideration in their designs. In a real application, however, there may be dozens of places where an object such as a Customer is referenced, and few programmers would be expected to locate each of these places. Thus, it is up to the designer to specifically catalog all of these references, and then to the project manager (not the individual programmer) to maintain this list of references, to make sure that all programmers know the places in the code that must be updated when each particular type of change is made, and to ensure that all the appropriate updates are implemented. If there are dozens of these structures to consider, and dozens of places to catalog for each structure, a reference list becomes difficult to maintain and use. This can become a major problem for the project manager and the individual programs, especially when new relationships are developed in the code, and programmers and managers are moved between projects. It is probably a fool's errant to even attempt to maintain this information, and so many times it is not maintained, and bugs crop up that a designer can conveniently attribute to the programmer, rather than to his/her own design.

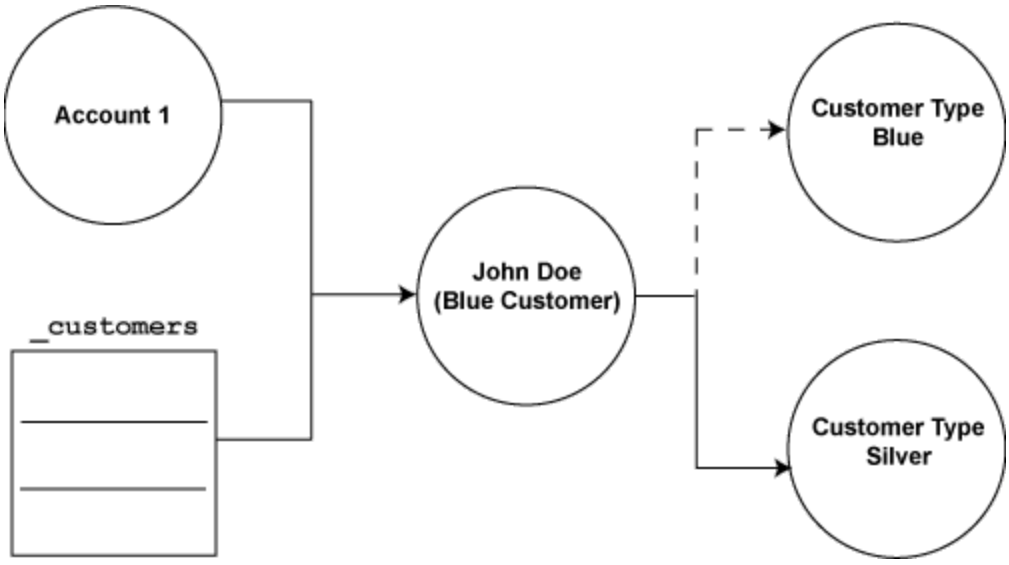
To make matters worse, the bugs created by this type of design error are also particularly insidious. For example, these bugs could occur in a production system for weeks before they are found. By then, a number of invalid objects could be

floating around the system, and trying to track down and fix the invalid data could, in itself, be difficult.

Also, the proper fix could be expensive in terms of runtime execution to fix all the bad references when changing the type. For example, in the worst case scenario for the bank, the entire _accounts collection would have to be walked in looking for a Customer who had changed types. If the bank had a large number of accounts, this would be a time-consuming and expensive operation. However, when designers use composition, they prevent all the problems that occur in changing an object's type. Using composition deters the bugs that would otherwise arise.

To see how composition prevents bugs, consider the composition design of this object, which was given in module 2 and is shown in figure 4.2:

Figure 4.2



References to the Customer Object in the Componsition Design of Bank

```
import java.util.ArrayList;
import java.util.Date;

interface CustomerType {
static float standardCharge = 25.00F;
public void processOverdraft(float amount) throws
InsufficentFundsException;
```

```
}

public class Customer {
private Name _name;
private Date _dateOfBirth;
CustomerType _custType;

ArrayList<Account> _accounts = new ArrayList<Account>();

private Customer(Name name, Date dateOfBirth, CustomerType custType) {
_name = name;
_dateOfBirth = dateOfBirth;
_custType = custType;
}

public static Customer getBlueCustomer(Name name, Date dateOfBirth) {
return new Customer(name, dateOfBirth, new BlueCustomer());
}

/* Method to retrieve name. */
public Name getName() {
return _name;
}

public void addAccount(Account account) {
_accounts.add(account);
}

public void processOverdraft(float amount) throws
InsufficentFundsException {
_custType.processOverdraft(amount);
}
}
```

In this design, the reference to the customer type still changes, but that reference is hidden inside the Customer object. The references to a Customer object in the _customers Map and the Account object are to the same Customer, and the Customer object hides the changed type. No external references ever need to change. This means that the Customer object can effectively change type without impacting the references in the rest of the program. This is extremely valuable behavior, and it is the reason for the existence of the State design pattern.

Another reason not to use classification when a type can change is that, if you need to record each time an object changes type, it is relatively easy with composition, and not so easy with classification. To understand why, again consider the Customer object. If you use composition, it is easy to set up a Collection of CustomerType objects and store each change to the CustomerType in that Collection. If, instead, you use classification, you need to store an entire Customer object, including all the data that did not change, to record any change in that Customer object.

There is a basic principle illustrated here. If many references can point to the same object, and it is expected that all the references consistently point to the same object, that reference should not be able to change. In the case discussed above, a proxy object must be created that (1) never changes its reference and (2) contains the one reference to the actual object of interest. Then, only one reference to an object needs to change to update all the referring objects.

Another problem with using classification when a history is needed is that, since a classification design results in a single instance of an object, it is impossible to separate the data for that object to just store the part that is needed for the history. When a history of only one part of an object is to be stored, classification designs are just too tightly coupled for a single object instance to be created.

Because of the reference problem and potential history problem in designs that use classification, and the fact that this problem does not occur in designs that use composition, there is seldom, if ever, a reason to use classification if your design will allow an object to change type. Unless there is some overriding criteria, it is best not to use classification when an object can change type.

Do Not Use Classification Unless You Really Want to Be an Instance of the Class You Are Extending

The second rule is not to use classification unless you really want to create an instance of the class you are extending. As was pointed out earlier, Java has true run time data typing. This means that the information about ancestor classes (for example, the parent class) is always available. Thus a programmer can write programs that rely on a class having a specific parent. However, in many cases there is not a good reason why the program should know what the class is that is extended. This is particularly true when a class is being used as a utility, to provide a service to another class.

A good example of a situation where a class is extended simply to provide a service is called representation inheritance (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m4-module-4/popups/popup4-ii-1.html?ou=693191) (or is-represented-by). Representation inheritance is shown in the following example. In this example, the instanceof operation shows that a Stack is a Vector, since it has extended Vector to represent the collection to store the elements in the Stack.

```
import java.util.*;

public class ShowStack {
public static void main(String args[]) {
Stack s = new Stack();
if (s instanceof Vector)
System.out.println("True");
}
}
```

This program raises the question, why does a Stack need to know whether or not it is a Vector? It is far more likely that this type of construction will be misused than that it will be used for a valid purpose. In fact, the larger question is, why should the type of data structure used to store the data in the Stack be available to the program at all?

In fact, the data structure used by a stack should probably not be available outside of the Stack class. That it is known to the implementing program that a Stack is a Vector is generally considered to be a bug in the Stack class as implemented in Java. One of the concrete results of the knowledge that a Vector is used by the Stack class is that Stack cannot be upgraded to use the new JDK 1.2 Collection classes, such as ArrayLists. However, other components that store collections of data, such as a Button, can be updated to use the new collections. The reason why Button can be upgraded and Stack cannot is that Button does not advertise what type of mechanism it uses to store its Collection data, so the Collection can be changed; a Stack advertises its Collection type (a Vector) through inheritance. Thus a Stack in Java is a Vector, and to change that fact changes what it is, which might cause some programs to fail.

For another example of this problem, consider the following simple GUI program. In it, the GUI has mixed Swing and Another Windowing Toolkit (AWT) components. AWT was the original GUI toolkit that came with Java starting with the J2SDK 1.0. Swing was an improved GUI toolkit that became available with J2SDK 1.3. The two GUI toolkits use mutually incompatible methods to access the underlying Operating System, and as anyone who has ever accidentally mixed these two types of components knows, the result is components that do not behave correctly in an application.

So why, if the toolkits are mutually incompatible, are the components from one toolkit allowed to interact with components from the other toolkit? This interaction occurs because components from both GUI toolkits extend java.awt.Component. The toolkits have some components in common; for example, the layout managers, which use the Component directly rather than separating the interface (which could have been consistent) from the type of component, which really does not have to be Component.

```
import java.awt.*;
import javax.swing.*;

public class ShowButton {
public static void main(String[] args) {
```

```
JFrame frame = new JFrame();
Container con = frame.getContentPane();

Button b = new Button("Wrong type");
con.add(b);

frame.pack();
frame.setVisible(true);
}
```

The basic principle displayed here is that, in Java, an object type is important. The contents of object say something about that object because Java keeps that information via runtime typing. Therefore when the extends clause (e.g. implement classification) is used in Java, something is said about the data type of the object, and so extends should be avoided unless that information is truly intended to be conveyed outside of the object.

Do Not Inherit Trouble Just Because It Is Free

One of the big arguments for the use of classification is that all the capability of the base class can be used by any subclass. In effect, this behavior is free to the subclass. Many programmers believe that, if something is free, you take it. But if what you are getting for free is trouble, it is a poor bargain.

For example, consider the follow example of using a Stack. Because Stack inherits from Vector, you get many methods for free, including an add() method. Although a programmer should know that the add() method should never be used for a stack (the only valid methods to manipulate the contents of a stack should be pop(), push(), and peek()), there are times in writing a program when a programmer simply types the wrong thing. A mistake of this type is shown in the program below, where a programmer[?] inadvertently uses the add() method instead of the push() method. This results in the stack storing the data in the wrong order.

```
import java.util.*;

public class ShowStack {
public static void main(String args[]) {
```

```
Stack<Integer> s = new Stack<Integer>();
s.push(1);
s.add(0,2);
System.out.println(s.pop());
System.out.println(s.pop());
}
}
```

Again, some programmers argue that this is not a problem with inheritance, but that the person using the Stack should be more careful. But why should it be easy to make an error that can easily be avoided? One of the main purposes of modern compilers and runtime systems is to recognize that programmers are human, and to protect them from silly and careless mistakes. This is especially true when all that the free behavior gets for you is the ability to call methods, such as the add() method above, which are seldom, if ever, used in a correct program. And, if such behavior is really needed, it can be accessed safely through Delegation anyway. Just because there is behavior in the Stack class that you can inherit for free, that does not mean that you should take it without considering the risks.

Another example of inheriting trouble can be noted when examining the Java GUI's, particularly Swing. Until recent versions of Swing, the following program would have resulted in a runtime error. The reason for this is that you cannot add components directly to a JFrame in Swing; you need to add them to one of the containers, such as the ContentPane, that are in the JFrame.

If you cannot add components directly to the JFrame, why then does the JFrame have an add() method? The add() method is present because it is inherited from Frame. But it serves no real purpose in the JFrame.

Recent versions of Java make any calls to the JFrame's add() method reference the ContentPane by default, but this does not change the fact that this is simply a kludge that was added to the system when it was determined that the add() method in JFrame was a mistake.

```
import java.awt.*;
import javax.swing.*;

public class ShowButton {
public static void main(String[] args) {
JFrame frame = new JFrame();
Container con = frame.getContentPane();

JButton b = new JButton("Something");
frame.add(b);

frame.pack();
frame.setVisible(true);
}
}
```

Another problem with inheriting unneeded functionality is that it results in bloated designs. For example, a JButton has over 200 methods, many of which you cannot and should not ever call.

This leads to the rule that you should not use classification simply to inherit the behavior of the parent—or, as stated above, do not inherit trouble simply because it is free. You should instead use composition and then use delegation to access the behavior you need.

Consider How the Class Will Be Stored and Used

Because classification couples classes to create one object, that object must represent a single entity. As a single entity, the object must reside in a single address space on a single computer. This can have a number of implications in, for example, a distributed application.

Consider the Bank application that used classification to design the Customer class. In the Bank application, an assumption was made that the code to check a Customer object for an overdraft and penalty would exist on the same computer as the code to store the base data for the Customer. Where was this assumption made? Because there is one object and that object has one reference, and a single

reference can only exist on a single computer, there is an implicit assumption that all checking for overdrafts will be done on the same computer that has stored the Customer object.

The compositional design, however, makes no such assumption about where the part of the Customer object that checks for overdraft and penalty exists. The CustomerType interface simply says that there will be a method that will be accessible on an object. Whether that object is on a local computer, or represents some remote service that exists on a totally separate computer, is not part of the description. The object that supplies the services for the CustomerType interface is completely decoupled from the Customer object, and thus can be distributed anywhere.

Even if, after the initial design is created, it is known that the CustomerType will be accessed on the local computer, it is still best to design the class using composition. This allows the design to adapt to changes in how the CustomerType is accessed. In general, unless there is something to be gained from using classification, it is always best to leave open options such as having the design become distributed.

Carefully Consider All Calls to Non-Final Virtual Methods

As was shown in module 1, virtual methods are methods that are always called on the actual type of the object. Since all methods in Java are virtual, if a non-final method is called, there is no guarantee that the method with that name in the current class will be called. If that method is overridden in any subclass, the overridden method will be called instead.

Virtual methods are often used by programmers to allow for the use of additional behavior in a subclass that is not provided in the base class. However, virtual methods should be used carefully because they allow for the possibility of abuse. If there is some set of conditions in the base class method on which the object relies, it is very possible for the child class to write the overridden method and remove that behavior. Once again, this cannot happen in a compositional design because the contained objects are delegated work, which is controlled by the encapsulating object.

Overriding virtual methods is not, in itself, a problem. The toString() method has been overridden frequently in these modules, and with good result. The place of virtual methods in a program, however, is something a programmer should be aware of and should use carefully.

Where this behavior does present a problem is in methods called by a constructor. The call to the constructor is supposed to be the first step in creating an instance of a class. But when a constructor calls a virtual method, and that virtual method is overridden in the subclass, the method of that subclass is executed before the object for that subclass has been constructed. This can lead to some strange behavior.

To view this behavior, consider the following simple classes. See that class B extends class A, and class A calls a method named method1() in its constructor. The method method1() simply sets the variable _var to 1. Subsequently, class B overrides method1(), but does not set the variable _var until it completes its constructor, when it sets _var to 2. So, what value of _var is printed out in the constructor for A? Zero (0). This is because all object variables are initialized to zero when the object is created, and no method has been called that changes this value before the constructor in A prints it out. This particular problem is very confusing, even to experienced programmers, because a properly constructed instance of A will have the value of _var set to 1, and a properly constructed instance of B will have a value of _var set to 2. During its construction of the object B, however, the base class constructor for A will have a transitory value of _var, which is invalid. If the constructor is relying on that value to be valid, the program will not work correctly.

```
public class ShowVirtual {
public static void main(String[] args) {
B b = new B();
}
}

class A {
int _var;
```

```
public A() {
System.out.println(_var);
}

public void method1() {
_var = 1;
}
}

class B extends A {
int _var;
public B() {
_var = 2;
}

public void method1() {
}
}
```

This type of problem arises frequently with virtual methods called from a constructor. It is impossible to ensure that the object is initialized before it is used, and in Java, that can lead to many strange results.

It is important to understand that this type of problem also occurs in concurrent programs. Concurrent programs are programs that use more than one thread of execution, and will be covered in more detail in module 5. However, concurrent programs are more prevalent than many programmers would like to believe. Any program that uses a GUI in Java is concurrent, and care must be taken when writing for concurrency to avoid program bugs.

In a concurrent program, if a thread is somehow created in a constructor, that thread has access to the data of the object even though that object might not yet have been constructed. This can lead to situations even stranger than the virtual method problem described above.

While the arguments here are not complete, we hope that they are sufficient to make the programmer carefully consider how classification through virtual methods can create situations where bugs can occur, and to carefully consider how to best implement and use virtual methods.

Know What You Are Sharing When You Share an Object Reference

Most Java programmers are aware that all objects in Java extend class Object. Most programmers, however, are unaware as to why objects in Java have this property, and the implications of the fact that all objects in Java are instances of class Object. For example, the class Object contains hidden data fields, such as a monitor, tha we will discuss in module 5.

When using classification to extend a class, it is once again important for the programmer to realize that he/she has created one, and only one, instance of the class Object. Thus, the base- and subclasses together share one common set of data for an Object. As we will show in module 5, the monitor field in the object is shared. This sharing greatly affects synchronized statements, which are used to safely ensure access to parts of an object.

Suppose, for example, a synchronized statement exists in a base class, and that base class is extended. If the subclass also chooses to use the synchronized statement, there will be only one monitor from the shared instance of Object that both classes have inherited. This means that the two classes will interfere with each others' use of the monitor. This interference can result in bugs that are extremely difficult to diagnose and fix, as will be seen in module 5.

In a composition design, these bugs simply do not occur because, when you create an object from a composite class, each object is separate. There is one object for each class. Therefore, each object is its own instance of class Object, and does not share the properties from class Object. Because no interactions occur between the classes, it is generally safer to simply ignore how the base class Object actually behaves. You do not need to consider how sharing this hidden data affects your application.

Once again, you should not couple classes into a single object unless you know what features you are coupling. When you extend any class in Java, you are also extending one instance of class Object. Composition is always safer then, because the objects are separate and do not share the data from class Object.

B. Choosing Composition or Classification

In the discussion so far, we have constantly stressed that composition should, in most circumstances, be favored over classification. The designs produced by composition tend to be less rigid and more stable and extensible. So, how do you choose which to use, classification or composition? In general, start your designs with composition, and use classification when you can show a discernable benefit.

This does not mean that classification should never be used. In fact, there are a number of situations in which classification designs are not simply better, but the application could not be effectively implemented without them. These designs tend to have the following four characteristics in common:

1. The objects do not change types. For example, immutable objects cannot change value, and therefore cannot change type.

2. The type is a true subtype. By this, we mean that the subtype is everything the base type is, with some extra functionality. Typically, this subtype relationship acts in a manner similar to a transaction.

3. There is functionality in the base type that can apply across all subtypes.

4. The type of the object, both the base type and subtype, needs to be exposed.

Two excellent examples of types that fit these criteria are java.lang.Exception and java.util.Event. In both cases, the objects created are immutable, so there is no concern about the type ever changing. In both cases, the types created are true subclass relationships; they are the base class plus extra information and functionality. In both cases, there is processing provided by the base type that is used in the subtypes (e.g., printStackTrace in the Exception class). Finally, in both cases, the program needs to know what the entire type is, both the base and parent

type. The base type is needed for generic handling (in particular, in the case of the Exception, to determine if the Exception is checked or unchecked), and the subtype is needed by the application which implements the Exception or Event.

There are a number of other examples in Java, so a programmer should always be open to the possibility of using classification. But in the end, unless there is a tangible benefit to using classification, composition should always be favored over classification.

III. Aggregation and Acquaintance

Designs that use composition are, generally speaking, safer than designs that use classification. However, this does not mean that there are not problems inherent to composition. In particular, the aptitude with which an object isolates information is critical to the design of a safe system. This section will deal with how to isolate data when using composition to create safer designs.

A. Encapsulation

Encapsulation (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m4-module-4/popups/popup4-iii-1.html?ou=693191) in OOP is the inclusion of one object within another thing so that the included object is not visible outside of the encapsulating object. For example, the following definition of a Name class is not encapsulated because the instance variables _firstName and _lastName are both visible and can be accessed from outside of the Name class.

```
public class Name {
public String _firstName;
public String _lastName;

public void setName(string firstName, string lastName) {
_firstName = firstName;
_lastName = lastName;
}
```

```
public String toString() {
return _firstName + " " + _lastName;
}
}
```

To encapsulate the instance variables, it is necessary to make them private so that they are hidden in the object, and to provide access to those hidden instance variables through methods. The following source code shows a properly encapsulated Name class.

```
public class Name {
private String _firstName;
private String _lastName;

public void setName(string firstName, string lastName) {
_firstName = firstName;
_lastName = lastName;
}

public String toString() {
return _firstName + " " + _lastName;
}
}
```

Encapsulation is the core mechanism responsible for ensuring managed access to an object and can be used in a number of ways. For example, it can be used to limit the range of values for a variable, as in the Person class below, where the age is limited to values between 0 and 125. This is an important addition to program safety.

```
public class Person {
private String _age;

private final boolean checkAge(int age) {
if (age > 0 && age < 125)
return true;
```

```
else
return false;
}

public Person(int age) {
if (checkAge(age)
_age = age;

else
_age = 0;
}

public void setAge(int age) {
if (checkAge(age)
_age = age;

else
_age = 0;
}
}
```

The ability to manage access to a variable has even stronger implications in program safety, as will be seen in the following sections.

If encapsulation is such a good thing, why doesn't a language such as Java make it required? There could be many reasons. One is that making a program work (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m4-module-4/popups/popup4-iii-2.html?ou=693191) sometimes requires that direct access to a variable be provided. The situations in which direct access is needed are often obscure, rare, and hard to explain. For example, some systems rely on a concept called *reflection*, which requires that the JVM actually have access to instance variables in the program. An example of reflection is used in Java Jini, which is an advanced distributed programming environment.

When the JVM needs to directly access instance variables, Java security requires that all variables in class be public variables. The reasons for this are beyond the scope of this class, but the idea is clear. There are cases, such as when reflection is being used, where a programmer cannot use encapsulation on instance variables. Generally speaking, however, a class should always implement encapsulation on its instance variables unless there is some overriding reason for not using it.

B. Aggregation and Acquaintance

Encapsulation is the first step toward isolating the instance variables inside a class. This principle will now be used to implement aggregation and acquaintance (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m4-module-4/popups/popup4-iii-3.html?ou=693191) . Gamma et al. (1995, 22–23) define aggregation and acquaintance as follows:

"Aggregation implies that one object owns or is responsible for another object. Generally we speak of an object having or *being part of* another object. Aggregation implies that an aggregate object and its owner have identical lifetimes."

"Acquaintance implies that an object merely *know of* another object. Sometimes acquaintance is called "association" or the "using" relationship. Acquainted objects may request operation of each other, but they aren't responsible for each other."

In the rest of this section, we will look at these distinctions between aggregation and acquaintance.

C. Acquaintance

Acquaintance represents a weaker control of one object over another, so we will explore it first. In an acquaintance relationship, one object knows about another object, so the encapsulating object stores a reference to the acquaintance object. However the acquaintance object has an existence (or has references) that can be modified outside of the encapsulating object. To understand this, consider the following example of a Car that encapsulates a Driver object. It is apparent from this relatively simple program that, while the Car object properly encapsulates

the Driver object, the Driver object can still be changed without the explicit knowledge of the encapsulating Car object. The ability to access and modify an object outside of the encapsulating object is an important design consideration because it implies that the Driver of the Car can be changed (or at least the Driver's name) without the Car object being aware of that change.

```
class Name {
private String _firstName;
private String _lastName;

public Name(String firstName, String lastName) {
_firstName = firstName;
_lastName = lastName;
}

public void setName(String firstName, String lastName) {
_firstName = firstName;
_lastName = lastName;
}
}

class Car {
private Name _driver;

public Car(Name driver) {
_driver = driver;
}
}

public class CarAndDriver {
public static void main(String[] args) {
Name name = new Name("Mario", "Andretti");
{
Car car = new Car(name);
name.setName("Dale", "Earnhardt");
```

```
} // <-- Note at this point car is out of scope, and
// can be garbage collected, but Name cannot.
}
}
```

The ability to modify an encapsulated object is important in many designs. There are designs that cannot exist without this property. But it is important for a programmer to recognize when an acquaintance relationship is being used, as the encapsulating object cannot be responsible for the state of the acquaintance object in a design. As we will show later, this relationship has even wider consequences, as programmers often do not even recognize that an acquaintance relationship has been implemented.

D. Aggregation

What is Aggregation?

In aggregation, the encapsulating object owns the aggregate object. What is meant by this is that the aggregate object cannot be changed outside of the encapsulating object, and thus, the encapsulating object can make guarantees about the state of the aggregate object. For example, consider the following implementation of a Car, where the _engine object is aggregated with the object. The _engine object is created in the Car constructor. Since no reference to the _engine object exists outside of this Carobject, all changes to the _engine object must be done through the Car. This is an important design consideration. It means that, unlike the previous Car class's Driver object, this Car class can make guarantees about the state of the _engine object.

```
class Engine {
private int _size;

public Engine(int size) {
_size = size;
}
}

class Car {
```

```
private Engine _engine;

public Car(int engineSize) {
_engine = new Engine(engineSize);
}
}

public class CarAndEngine {
public static void main(String[] args) {
Car car = new Car(350);
} //<- Note that car goes out of scope, and so does _engine,
// So both can be garbage collected.
}
```

Note that there are other possible ways to create an _engine object for a Car and still use aggregation. One is to pass an Engine object into the Car constructor, or any other method, and simply then clone the Engine object. Since this means that the Engine object stored in the Car class is a different object from the one passed in as a parameter, this Engine object now has no reference outside of the encapsulating object, resulting in an aggregation relationship. Note that this same technique must be followed if an aggregate object is ever returned from a method in the encapsulating object. If a clone is not made, the reference has escaped the encapsulating object, and the programmer has created a possibly incorrect acquaintance relationship.

```
class Engine implements Cloneable {
private int _size;

public Engine(int size) {
_size = size;
}

public Engine clone() {
return new Engine(this._size);
}
```

```
}

class Car {
private Engine _engine;

public Car(Engine engine) {
_engine = engine.clone();
}

public Engine getEngine() {
return _engine.clone();
}
}

public class CarAndEngine {
public static void main(String[] args) {
Engine engine = new Engine(350);
{
Car car = new Car(engine);
} //<- Note that car goes out of scope, and so
// does _engine, so both can be garbage
// collected. However the engine created
// in this main method is still in scope
// and cannot yet be garbage collected.
}
}
```

One final note on when an aggregate object can be garbage collected: in these programs, because there is no reference to the aggregate object outside of the encapsulating object, when the encapsulating Car object is garbage collected, the only reference to the aggregate _engine object should be removed so that the aggregate object can also be garbage collected. It would seem logical to interpret Gamma's words, "Aggregation implies that an aggregate object and its owner have

identical lifetimes," to refer to the fact the aggregate _engine object was created at the same time as the encapsulating Car object. Whether this is what Gamma truly meant will be the subject of the next section.

Aggregation and Lifetimes of Objects

The terminology used in Gamma's definition, that an "aggregate object and its owner have identical lifetimes," is used to make a point about aggregates, but it is misleading. The point is that an aggregate object is completely owned by the encapsulating object, and so when the encapsulating object goes out of scope, the aggregate object (typically) also goes out of scope, because it is owned by (or has no reference outside of) the encapsulating object. Thus, when an encapsulating object is garbage collected, any aggregated objects are (generally) garbage collected. An acquaintance object, on the other hand, cannot be garbage collected when the encapsulating object is garbage collected because, by definition, an acquaintance object has a life outside of the encapsulating object.

Gamma's definition implies that no reference to an aggregated object exists outside of the encapsulating object. This is true, but the statement that an "aggregate object and its owner have identical lifetimes" is too limiting. For example, consider the following simple Car class, where an Engine can be replaced in the Car. In this Car class, neither the old Engine nor the new Engine objects have a reference outside of the Car, and thus, they are, appropriately, aggregation relationships with the Car object. This is properly aggregation because the Car object can make guarantees about the behavior of both Engine objects, yet neither of the Engines has the same lifetime as the Car object.

```
class Engine implements Cloneable {
private int _size;

public Engine(int size) {
_size = size;
}

public Engine clone() {
return new Engine(this._size);
```

```
}
}

class Car {
private Engine _engine;

public Car(Engine engine) {
_engine = engine.clone();
}

public Engine getEngine() {
return _engine.clone();
}

public void replaceEngine(Engine engine) {
_engine = engine.clone();
}
}
```

An even weaker aggregation relationship exists where the object to be aggregated is immutable. For example, consider the following definition of a Car class where, in the Car constructor, a String parameter for the modelType variable is provided. Obviously, the String object has a reference outside of the Car; however, because String is immutable, there is no problem in simply setting the modelType instance variable to the same reference as the parameter. To have to say "_modelType = new String(modelType);" is simply overkill in this instance.

```
class Car {
private String _modelType;

public Car(String modelType) {
_modelType = modelType;
}
}
```

E. When to use Aggregation and Acquaintance

This all leads to the question of when to use aggregation and acquaintance, and when to make the aggregate object immutable. To determine the answer, we must first consider that a type that implements an immutable object is an aggregation relationship that has additional constraints; specifically that the data values cannot be changed. Thus, immutable objects are the safest objects to use in a program. Second, we must realize that, in acquaintance relationships, references are available outside of the encapsulating object; therefore, aggregation relationships are safer than acquaintance relationships.

So, what criteria can be used to choose which relationship to implement? A good rule of thumb is to start with the most stringent (safest) design, and to loosen these constraints when the need arises. The safest course is to first assume that all classes in a system should be immutable. Obviously, this cannot be true, but classes should be changed from ones that are immutable to ones that are aggregate only when the need arises. As the design continues to evolve, it will become apparent that certain classes need to be acquaintance; however, only when the need arises should a class that uses an aggregate relationship be made into one that uses an acquaintance relationship.

F. Recognizing Acquaintance Relationships

One major problem in implementing classes is that it is often easy for a relationship intended to be an aggregation relationship to degenerate accidentally into one that is an acquaintance relationship. Consider the Customer class used in the case study in module 3. In the Customer class, there is an instance variable _accounts that is a collection of Account objects owned by this Customer object. When the GUI was implemented, there was a need to display these Account objects, so a getAccounts() method was innocently added that simply returned this Collection. The problem with this addition is that now, the _accounts Collection is no longer an aggregate of the Customer object. Any programmer (either through ignorance, haste, or malicious intent) can access this Collection and modify it. So, although adding the getAccounts() method did not result in any immediately apparent program errors, the addition must be considered a bug in the program.

How can this problem be avoided? Earlier, we saw that when you want to pass data out of a method without violating an aggregation relationship, you can simply clone the object(s) that you wish to return. This can be done as follows:

```
public Collection<Account> getAccounts() {
return (Collection<Account>)_accounts.clone();
}
```

This method works well for relatively small Collections that are not frequently accessed, but if the number of items grows in this Collection, cloning it each time it must be returned can become prohibitively expensive. Java has provided a way to wrapper a Collection so that it cannot be modified. You can create this wrapper by calling the Collections.unmodifiableCollection() method, as illustrated in the code fragment below.

```
public Collection<Account> getAccounts() {
return (Collection<Account>)
(Collections.unmodifiableCollection(_accounts));
}
```

Note that this Collection technically violates the rule that, for an aggregate object, a reference to the object should not exist outside of the encapsulating object. However, the procedure of using a wrapper can provide a significant performance increase over cloning, and it does keep the aggregate object safe. The wrapper achieves the necessary safety constraints of an aggregate object at a much lower cost than by cloning.

The above example illustrates why it is important to understand why rules are created, and to not simply memorize the rules and mechanically apply them. The unmodified collection does, indeed, violate the rule about aggregate data having the same lifetime as the encapsulating object. But blindly following this rule could result in a program that is inefficient and not useful.

If one idea is taken away from this module (and indeed this class), it should be that, in all design decisions, creating a safe and effective design is more important than following a set of rules. In the final analysis, a programmer has to do what he/she

must to field a program. If a programmer understands why certain rules exist, he or she can apply the principles behind the rules to make sound decisions that will enhance and safeguard a program.

The Java™ Tutorials

Trail: Creating a GUI With Swing

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Lesson: Writing Event Listeners

Examples Index

This lesson gives you details about writing event listeners. You might not need to read this section. Your first source of information for event should be the how-to section for the component in question. Each component's section shows code for handling the events most commonly needed when implementing the component. For example, How to Use Check Boxes shows you how to handle mouse clicks on check boxes using an item listener.

Some Simple Event-Handling Examples

The programs in this section illustrate events and event handling.

General Information about Writing Event Listeners

This section provides information that is useful for handling all types of events. One of the topics includes information on using adapters and inner classes to implement event handlers.

Listeners Supported by Swing Components

This is *the* place to find out which Swing components can fire which kinds of events. You might want to bookmark this section so you can easily find its quick-reference table.

Implementing Listeners for Commonly Handled Events

This section has detailed information and examples of writing each common kind of event listener.

Listener API Table

This section features a quick-reference table that shows each listener, its adapter class (if any), and its methods.

Solving Common Event-Handling Problems

If you are having some hard-to-debug problems related to handling events, you might find the solution here.

Questions and Exercises

Try these questions and exercises to test what you have learned in this lesson.

If you are interested in using JavaFX to create your GUI, see Handling JavaFX Events.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Previous Lesson
Next page: Introduction to Event Listeners

| Ad Choices

The Java™ Tutorials

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Trail: Creating a GUI With Swing

Also known as *The Swing Tutorial*

This trail tells you how to create graphical user interfaces (GUIs) for applications and applets, using the Swing components. If you would like to incorporate JavaFX into your Swing application, please see Integrating JavaFX into Swing Applications.

Getting Started with Swing is a quick start lesson. First it gives you a bit of background about Swing. Then it tells you how to compile and run programs that use Swing components.

Learning Swing with the NetBeans IDE is the fastest and easiest way to begin working with Swing. This lesson explores the NetBeans IDE's GUI builder, a powerful feature that lets you visually construct your Graphical User Interfaces.

Using Swing Components tells you how to use each of the Swing components — buttons, tables, text components, and all the rest. It also tells you how to use borders and icons.

Concurrency in Swing discusses concurrency as it applies to Swing programming. Information on the event dispatch thread and the SwingWorker class are included.

Using Other Swing Features tells you how to use actions, timers, and the system tray; how to integrate with the desktop class, how to support assistive technologies, how to print tables and text, how to create a splash screen, and how to use modality in dialogs.

Laying Out Components Within a Container tells you how to choose a layout manager, how to use each of the layout manager classes the Java platform provides, how to use absolute positioning instead of a layout manager, and how to create your own layout manager.

Modifying the Look and Feel tells you how to specify the look and feel of Swing components.

Drag and Drop and Data Transfer tells you what you need to know to implement data transfer in your application.

Writing Event Listeners tells you how to handle events in your programs.

Performing Custom Painting gives you information on painting your own Swing components. It discusses painting issues specific to Swing components, provides an overview of painting concepts, and has examples of custom components that paint themselves.

Other UI-Related Trails

Although this is the main trail for learning about GUIs, it isn't the only trail with UI-related information.

- 2D Graphics, which describes the 2D graphics features available in the JDK.
- Sound, which discusses the sound capabilities available in the JDK.
- Java Applets, which describes API available only to applets.
- Essential Java Classes, which covers many topics, including properties and the standard I/O streams.
- The JavaFX Documentation, which describes how to build UIs with JavaFX.
- The Bonus trail contains Full-Screen Exclusive Mode API, a lesson that describes how to use API introduced in v1.4 to render graphics directly to the screen.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

| Ad Choices

Previous page: Beginning of Tutorial
Next page: Getting Started with Swing

| Ad Choices

Module 3: Creating an Application

The classes created in module 2 will now be used to create a simple application. This application is not complete but instead is used to show some of the principles involved in actually implementing a GUI application. The changes needed to the classes from module 2, as well as the actual implementation of the GUI, will be illustrated in the following sections.

I. Changes to the Bank class

The class that is most changed in this application is the Bank class. These changes are explained below.

Programs/bank/Bank.java (https://leocontent.umgc.edu/content/dam/permalink/9f390320-d635-4c30-9777-07ee9156157b.html?ou=693191)

A. Using Maps to Store Data

The first change that was made to Bank was to take _accounts and _customers collections and change them to implement Java Maps. The reason for doing this is that these two structures are used to find an object, such as an Account or Customer, given a key (account number or name). This change not only allows the data to be found by a key value, but also allows the keys to be the only immutable part of the data object.

In this case, both of these changes are implemented using HashMaps, but as is good practice, the actual type of Map that is used is specified only as a Map, which allows it to be changed later if the need arises (for example, if a sorted list of clients is needed). The key for the Accounts is an Integer, and so autoboxing is used in the application. The key for the Customer is the customer name, so the Name class was changed to make it immutable. The new Name class is referenced below. Note the

use of the final modifier in the name class for the String variables _firstName and _lastName. It should be obvious that final in Java is not used simply to define constants.

In this case, the final modifier simply says that the value must be set before the object is finished being constructed, so we can set the actual values in the constructor. After the object is constructed, the variables cannot be changed. The fact that the string itself is immutable ensures that these variables cannot be changed, and thus a Name object is immutable.

Programs/bank/Name.java (https://leocontent.umgc.edu/content/dam/permalink/c8637962-dee3-4c89-a8c5-a3e2d16de801.html?ou=693191)

Even though it is immutable, whether it was a good decision to make the key to the Customer the Name is explored in the self-assessment questions.

B. The Ability to Save and Restore the Bank Was Added

The next major change to the program was that the ability to save and restore the bank was added. Saving the Bank is accomplished by writing the _accounts and _customers Maps to an ObjectOutputStream. Note that both streams can be written to the same file, as can the integer variable used to determine the account number for the next account. The Bank can then be reconstituted by simply reading these two Map objects using an ObjectInputStream. This example shows the power of ObjectStreams.

Note that in module 1 we wrote the Account class so that it would keep track of the _nextAccountNumber in a static variable inside the Account class itself. This worked well because it did not need to be visible outside of the Account class. However, now when a Bank is saved and restored, we must read, and worse, assign this number. So methods were added to the Account class to manipulate the _nextAccountNumber. The effect of this decision is explored in the self-assessment questions.

Serializing classes

Because ObjectStreams are used to save the data, all of the classes for objects that are to be written to the ObjectStream must be declared to implement Serializable. This declaration is generally needed for all classes, and simply involves having these classes implement Serializable.

Note that when these classes are compiled, a compiler warning is generated that the variable serialVersionUID is not declared. This warning was left in most of the classes to illustrate what happens when it is left out; however, it was fixed in the BlueCustomer class to show how to fix this warning. The BlueCustomer class is referenced below.

Programs/bank/BlueCustomer.java (https://leocontent.umgc.edu/content/dam/permalink/fb4b90ab-92aa-44e9-898a-c0680c198f37.html?ou=693191)

The reason for this warning is that if ever a class is changed to add new data items to the class, objects that were written with a previous version of this class will no longer be valid. If all that is changed, however, is the functionality of a method, then the objects created with the old version of this class are still valid with the new version. The serialVersionUID variable is used to give the programmer control over determining if an object that was generated and stored to an ObjectStream is still valid to be used with a new version of a class. If the serialVersionUID is set, it is up to the programmer to ensure that the class is still valid or that the variable is updated. If the variable is not set, the compiler generates a new random number each time the class is compiled. If the serialVersionUID does not match between the object written to the ObjectStream and the current version of the class, an exception is thrown when the object is read.

A complete new set of the files needed for this system is given below. Note that generally the changes have been limited to adding the implement Serializable clause to each class.

Programs/bank/Account.java (https://leocontent.umgc.edu/content/dam/permalink/6c5abf38-0fc5-4fd7-811b-cadf8e43ded0.html?ou=693191)
Programs/bank/CheckingAccount.java

(https://leocontent.umgc.edu/content/dam/permalink/dc3857f0-7d96-43a8-aa42-596e8cc55f4f.html?ou=693191)
Programs/bank/SavingsAccount.java
(https://leocontent.umgc.edu/content/dam/permalink/f0aebae1-3aa7-4557-b53e-b535f2ec91ac.html?ou=693191)
Programs/bank/Customer.java
(https://leocontent.umgc.edu/content/dam/permalink/fe6602a2-c206-4d28-ad8b-03e3c592298e.html?ou=693191)
Programs/bank/GoldCustomer.java
(https://leocontent.umgc.edu/content/dam/permalink/02a6bc4b-b4f4-45ec-9144-fc145c2de707.html?ou=693191)
Programs/bank/SilverCustomer.java
(https://leocontent.umgc.edu/content/dam/permalink/e76e9bb1-ced4-4135-8c67-929128535797.html?ou=693191)
Programs/bank/BankException.java
(https://leocontent.umgc.edu/content/dam/permalink/8f5da5b8-c23f-4bc4-9930-6df368022e07.html?ou=693191)
Programs/bank/AccountAccessException.java
(https://leocontent.umgc.edu/content/dam/permalink/98ab4fed-a698-4bdc-93fc-62c4e9a9d636.html?ou=693191)
Programs/bank/CustomerAccessException.java
(https://leocontent.umgc.edu/content/dam/permalink/e852b4f0-336c-4701-9286-5310ab09c985.html?ou=693191)
Programs/bank/InsufficentFundsException.java
(https://leocontent.umgc.edu/content/dam/permalink/f7e1928b-a6ab-488d-a087-6d76799a752e.html?ou=693191)

The Unit Testing Program

Note that the unit testing program, TestBank.java, which was used in module 1, is invaluable as we make changes to the Bank to test new features as they are added. Too often new features break old ones, so unit tests for a class are often considered as important as the class itself. In this module, unit tests are written by hand. However, in industry, there are a number of unit testing tools. The two best in the Java world, JUnit and TestNG, are both free.

Always remember to keep your unit tests when developing a system and to keep them up to date. They can save countless hours of debugging of the overall system when a unit- level problem is introduced.

One interesting problem in the testing was that to be able to test the save and restore methods, we had the Bank return a Collection to the _customers and _accounts Maps. Whether or not this was a good decision will be the subject of a discussion in module 4.

Programs/TestBank.java (https://leocontent.umgc.edu/content/dam/permalink/2d4325fe-b1c8-4c7c-83ae-542a35dd9c9c.html?ou=693191)

II. Writing the GUI

Finally, we are ready to write the GUI. Note that the GUI includes the JList, JScrollPane, and JPanel that were discussed earlier in this module.

The most important feature to be considered in the GUI, however, is that no application logic is present. Everything in the GUI is about retrieving and displaying the data on the forms. All the application logic is performed through calls to the Bank application. This is the crux of "n-tiered" architectures, the most common of which is called a *three-tiered architecture.*

In three-tiered architecture, there are three separate layers to the application. The outermost layer, the presentation layer, should be concerned only with formatting, displaying, and inputting data. No application logic should exist in this layer. The middle layer(s) is where the business logic should reside. Finally, the innermost layer should be concerned with the data, such as databases.

These layers access each other only through messages that are passed between the tiers. Each layer in the application should know nothing about the other layers except how to send and receive messages from them. This isolation allows each layer to concentrate on specific issues for that layer. This type of architecture allows the application to change more easily, because changes are more isolated. It

also improves security, because only the relevant information for a layer is available. For example, the presentation layer cannot directly access the data layer, which allows the data layer to be hidden from the end user.

Note that our application is really only two-tiered, because there is no data access layer and all data is kept in the _accounts and _customers maps. In a business application, these data should be moved out of the middle tier. But the concept of not allowing business logic access, or much worse, actual access to the data, in the presentation layer is something that should be observed.

There is one more potential situation to consider. To display the accounts owned by a customer in the GUI, a reference to the Collection for the account for a Customer was actually passed to the presentation layer using the getAccounts() method. This passing is similar to the returning of Collections referring to the _customers and _accounts for the Bank discussed in the previous section. Once again, this issue will be discussed in module 4.

Programs/BankApplication.java (https://leocontent.umgc.edu/content/dam/permalink/45713a1a-e754-4cb8-bf0c-76d79b6b63fb.html?ou=693191)

The application is now complete. To compile the application, type "javac BankApplication.java", and to run it, type "java BankApplication".

Make sure when you compile these programs that the "bank" package is in your CLASSPATH, and that you compile from a separate directory that contains the TestBank and BankApplication. Please note that you must correctly put the files for the "bank" package in a separate directory named "bank", and this directory must be different from the files TestBank and BankApplication.

The Java™ Tutorials

Trail: Essential Java Classes

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Lesson: Concurrency

Computer users take it for granted that their systems can do more than one thing at a time. They assume that they can continue to work in a word processor, while other applications download files, manage the print queue, and stream audio. Even a single application is often expected to do more than one thing at a time. For example, that streaming audio application must simultaneously read the digital audio off the network, decompress it, manage playback, and update its display. Even the word processor should always be ready to respond to keyboard and mouse events, no matter how busy it is reformatting text or updating the display. Software that can do such things is known as *concurrent* software.

The Java platform is designed from the ground up to support concurrent programming, with basic concurrency support in the Java programming language and the Java class libraries. Since version 5.0, the Java platform has also included high-level concurrency APIs. This lesson introduces the platform's basic concurrency support and summarizes some of the high-level APIs in the `java.util.concurrent` packages.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Previous Lesson
Next page: Processes and Threads

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Processes and Threads

In concurrent programming, there are two basic units of execution: *processes* and *threads*. In the Java programming language, concurrent programming is mostly concerned with threads. However, processes are also important.

A computer system normally has many active processes and threads. This is true even in systems that only have a single execution core, and thus only have one thread actually executing at any given moment. Processing time for a single core is shared among processes and threads through an OS feature called time slicing.

It's becoming more and more common for computer systems to have multiple processors or processors with multiple execution cores. This greatly enhances a system's capacity for concurrent execution of processes and threads — but concurrency is possible even on simple systems, without multiple processors or execution cores.

Processes

A process has a self-contained execution environment. A process generally has a complete, private set of basic run-time resources; in particular, each process has its own memory space.

Processes are often seen as synonymous with programs or applications. However, what the user sees as a single application may in fact be a set of cooperating processes. To facilitate communication between processes, most operating systems support *Inter Process Communication* (IPC) resources, such as pipes and sockets. IPC is used not just for communication between processes on the same system, but processes on different systems.

Most implementations of the Java virtual machine run as a single process. A Java application can create additional processes using a `ProcessBuilder` object. Multiprocess applications are beyond the scope of this lesson.

Threads

Threads are sometimes called *lightweight processes*. Both processes and threads provide an execution environment, but creating a new thread requires fewer resources than creating a new process.

Threads exist within a process — every process has at least one. Threads share the process's resources, including memory and open files. This makes for efficient, but potentially problematic, communication.

Multithreaded execution is an essential feature of the Java platform. Every application has at least one thread — or several, if you count "system" threads that do things like memory management and signal handling. But from the application programmer's point of view, you start with just one thread, called the *main thread*. This thread has the ability to create additional threads, as we'll demonstrate in the next section.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Concurrency
Next page: Thread Objects

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Thread Objects

Each thread is associated with an instance of the class `Thread`. There are two basic strategies for using `Thread` objects to create a concurrent application.

- To directly control thread creation and management, simply instantiate `Thread` each time the application needs to initiate an asynchronous task.
- To abstract thread management from the rest of your application, pass the application's tasks to an *executor*.

This section documents the use of `Thread` objects. Executors are discussed with other high-level concurrency objects.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Processes and Threads
Next page: Defining and Starting a Thread

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: Thread Objects

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Defining and Starting a Thread

An application that creates an instance of `Thread` must provide the code that will run in that thread. There are two ways to do this:

- *Provide a `Runnable` object.* The `Runnable` interface defines a single method, `run`, meant to contain the code executed in the thread. The `Runnable` object is passed to the `Thread` constructor, as in the `HelloRunnable` example:

```
public class HelloRunnable implements Runnable {

    public void run() {
        System.out.println("Hello from a thread!");
    }

    public static void main(String args[]) {
        (new Thread(new HelloRunnable())).start();
    }

}
```

- *Subclass `Thread`.* The `Thread` class itself implements `Runnable`, though its `run` method does nothing. An application can subclass `Thread`, providing its own implementation of `run`, as in the `HelloThread` example:

```
public class HelloThread extends Thread {

    public void run() {
        System.out.println("Hello from a thread!");
    }

    public static void main(String args[]) {
        (new HelloThread()).start();
    }

}
```

Notice that both examples invoke `Thread.start` in order to start the new thread.

Which of these idioms should you use? The first idiom, which employs a `Runnable` object, is more general, because the `Runnable` object can subclass a class other than `Thread`. The second idiom is easier to use in simple applications, but is limited by the fact that your task class must be a descendant of `Thread`. This lesson focuses on the first approach, which separates the `Runnable` task from the `Thread` object that executes the task. Not only is this approach more flexible, but it is applicable to the high-level thread management APIs covered later.

The `Thread` class defines a number of methods useful for thread management. These include `static` methods, which provide information about, or affect the status of, the thread invoking the method. The other methods are invoked from other threads involved in managing the thread and `Thread`

| Ad Choices

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Thread Objects
Next page: Pausing Execution with Sleep

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: Thread Objects

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Pausing Execution with Sleep

`Thread.sleep` causes the current thread to suspend execution for a specified period. This is an efficient means of making processor time available to the other threads of an application or other applications that might be running on a computer system. The `sleep` method can also be used for pacing, as shown in the example that follows, and waiting for another thread with duties that are understood to have time requirements, as with the `SimpleThreads` example in a later section.

Two overloaded versions of `sleep` are provided: one that specifies the sleep time to the millisecond and one that specifies the sleep time to the nanosecond. However, these sleep times are not guaranteed to be precise, because they are limited by the facilities provided by the underlying OS. Also, the sleep period can be terminated by interrupts, as we'll see in a later section. In any case, you cannot assume that invoking `sleep` will suspend the thread for precisely the time period specified.

The `SleepMessages` example uses `sleep` to print messages at four-second intervals:

```
public class SleepMessages {
    public static void main(String args[])
        throws InterruptedException {
        String importantInfo[] = {
            "Mares eat oats",
            "Does eat oats",
            "Little lambs eat ivy",
            "A kid will eat ivy too"
        };

        for (int i = 0;
             i < importantInfo.length;
             i++) {
            //Pause for 4 seconds
            Thread.sleep(4000);
            //Print a message
            System.out.println(importantInfo[i]);
        }
    }
}
```

Notice that `main` declares that it `throws InterruptedException`. This is an exception that `sleep` throws when another thread interrupts the current thread while `sleep` is active. Since this application has not defined another thread to cause the interrupt, it doesn't bother to catch `InterruptedException`.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

| Ad Choices

Previous page: Defining and Starting a Thread
Next page: Interrupts

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: Thread Objects

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Interrupts

An *interrupt* is an indication to a thread that it should stop what it is doing and do something else. It's up to the programmer to decide exactly how a thread responds to an interrupt, but it is very common for the thread to terminate. This is the usage emphasized in this lesson.

A thread sends an interrupt by invoking `interrupt` on the `Thread` object for the thread to be interrupted. For the interrupt mechanism to work correctly, the interrupted thread must support its own interruption.

Supporting Interruption

How does a thread support its own interruption? This depends on what it's currently doing. If the thread is frequently invoking methods that throw `InterruptedException`, it simply returns from the `run` method after it catches that exception. For example, suppose the central message loop in the `SleepMessages` example were in the `run` method of a thread's `Runnable` object. Then it might be modified as follows to support interrupts:

```
for (int i = 0; i < importantInfo.length; i++) {
    // Pause for 4 seconds
    try {
        Thread.sleep(4000);
    } catch (InterruptedException e) {
        // We've been interrupted: no more messages.
        return;
    }
    // Print a message
    System.out.println(importantInfo[i]);
}
```

Many methods that throw `InterruptedException`, such as `sleep`, are designed to cancel their current operation and return immediately when an interrupt is received.

What if a thread goes a long time without invoking a method that throws `InterruptedException`? Then it must periodically invoke `Thread.interrupted`, which returns `true` if an interrupt has been received. For example:

```
for (int i = 0; i < inputs.length; i++) {
    heavyCrunch(inputs[i]);
    if (Thread.interrupted()) {
        // We've been interrupted: no more crunching.
        return;
    }
}
```

In this simple example, the code simply tests for the interrupt and exits the thread if one has been received. In more complex applications, it might make more sense to throw an `InterruptedException`:

```
if (Thread.interrupted()) {
    throw new InterruptedException();
```

| Ad Choices

This allows interrupt handling code to be centralized in a `catch` clause.

The Interrupt Status Flag

The interrupt mechanism is implemented using an internal flag known as the *interrupt status*. Invoking `Thread.interrupt` sets this flag. When a thread checks for an interrupt by invoking the static method `Thread.interrupted`, interrupt status is cleared. The non-static `isInterrupted` method, which is used by one thread to query the interrupt status of another, does not change the interrupt status flag.

By convention, any method that exits by throwing an `InterruptedException` clears interrupt status when it does so. However, it's always possible that interrupt status will immediately be set again, by another thread invoking `interrupt`.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Pausing Execution with Sleep
Next page: Joins

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: Thread Objects

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Joins

The `join` method allows one thread to wait for the completion of another. If `t` is a `Thread` object whose thread is currently executing,

```
t.join();
```

causes the current thread to pause execution until `t`'s thread terminates. Overloads of `join` allow the programmer to specify a waiting period. However, as with `sleep`, `join` is dependent on the OS for timing, so you should not assume that `join` will wait exactly as long as you specify.

Like `sleep`, `join` responds to an interrupt by exiting with an `InterruptedException`.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Interrupts
Next page: The SimpleThreads Example

| Ad Choices

Module 5: Concurrency in Java

Topics

1. What is Concurrent Programming? (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m5-module-5/s3-commentary.html?ou=693191#I)
2. Safety and Liveness (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m5-module-5/s3-commentary.html?ou=693191#II)
3. Designing Active and Passive Objects (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m5-module-5/s3-commentary.html?ou=693191#III)

I. What is Concurrent Programming?

This module discusses concurrent programming, particularly as it relates to the design of objects. But before we begin this discussion, it is first necessary for you to understand what concurrent programming is, and the effects of concurrency on a program.

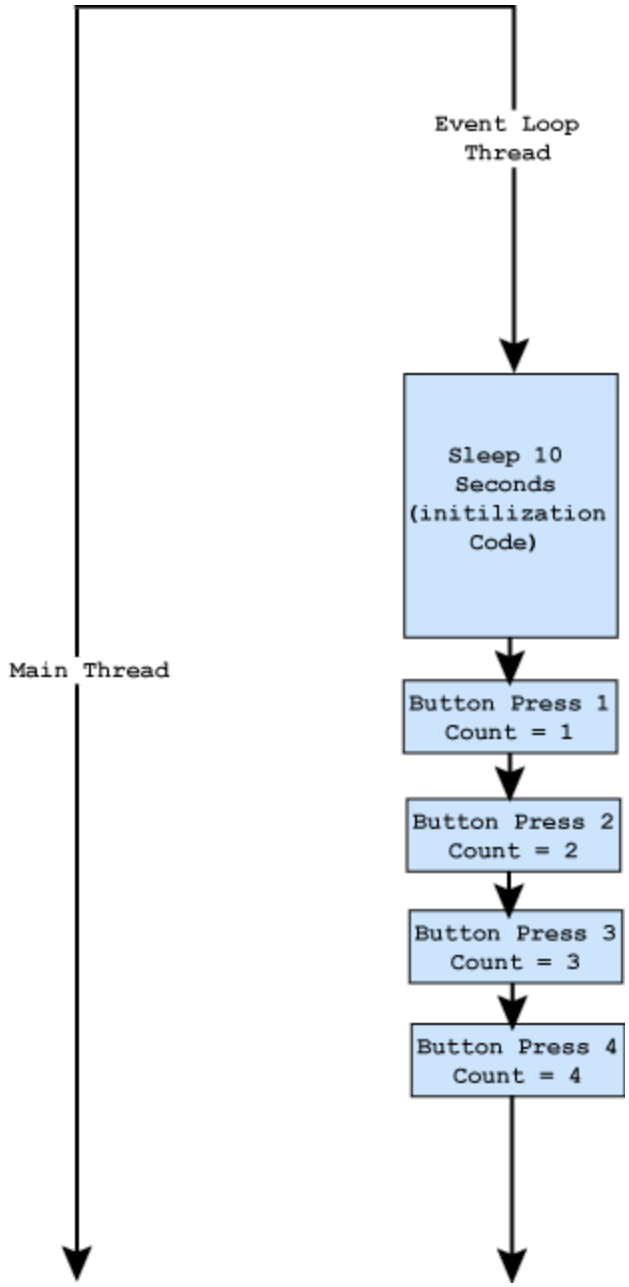
You may know that concurrent programming involves two or more threads of execution occurring at the same time. You may also be aware that concurrent programming involves using Java Threads. You may, however, not know that any program written in Java involves concurrency and Threads (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m5-module-5/popups/popup5-i-1.html?ou=693191) . The concurrency in most Java programs is hidden from the programmer, and generally does not affect a program. That does not mean, though, that the concurrency cannot result in unusual behavior or errors in a program.

To begin the discussion of concurrency, we will examine the reason for studying concurrent programming and give some simple examples of concurrent programs that all students have implemented. Then, we will discuss programming errors specific to concurrent programming, along with methods of finding and fixing these problems. Finally, we will provide an example of how to design a concurrent program using objects.

A. Why Study Concurrent Programming?

Most Java programmers, some with many years of experience, believe that they have never written concurrent programs. In fact, programs in which the programmer explicitly uses threads in her or his program are rare. However, threads are common, and actually nearly universal, in Java programs. To illustrate this point, run the two programs shown below. The first shows a JButton and a variable counter displayed in a JTextField. The first time you press the JBbutton, the program waits 10 seconds, then sets the counter in the JTextField to 1. Subsequently, the counter simply adds one to the value each time you press theJButton, keeping count of the number of times the JButton is pressed. This program is illustrated in figure 5.1. As this figure shows, there are two threads; however, because all the relevant execution is done in the Event Loop thread, the program runs correctly.

Figure 5.1



Button Program with All Logic in Event Loop

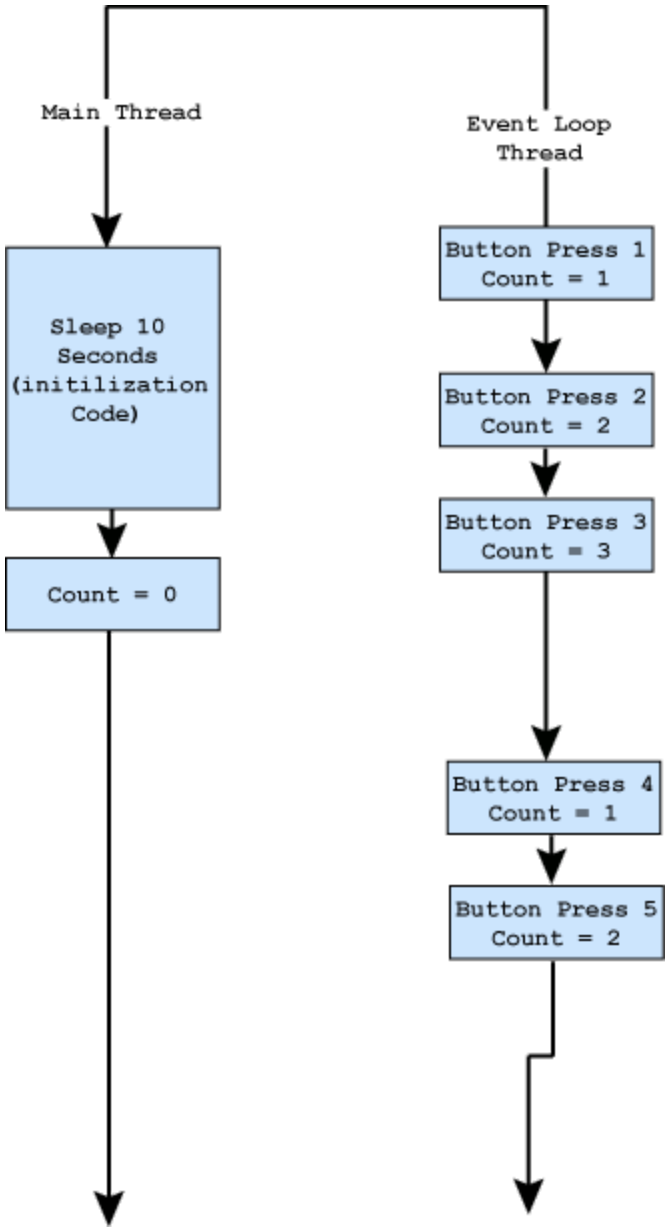
```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class GUIExample1 {
private static int counter = 0;

public static void main(String args[]) {
JFrame frame = new JFrame("Counter");
```

```
Container con = frame.getContentPane();
con.setLayout(new FlowLayout());

con.add(new JLabel("Counter"));
final JTextField counterText = new JTextField("0 ");
con.add(counterText);

JButton counterButton = new JButton("Increment");
counterButton.addActionListener(new ActionListener() {
public void actionPerformed(ActionEvent ae) {
try {
if (counter == 0) {
Thread.sleep(10000);
counter = 1;
}
else {
counter++;
}

counterText.setText("" + counter);
} catch (InterruptedException ie) {
}
}
});
con.add(counterButton);

JButton exitButton = new JButton("Exit");
exitButton.addActionListener(new ActionListener() {
public void actionPerformed(ActionEvent ae) {
System.exit(0);
}
});
con.add(exitButton);

frame.pack();
```

```
frame.setVisible(true);
}
}
```

Contrast this first program with the following program, illustrated in figure 5.2. In this program, the counter is initialized after a 10-second wait (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m5-module-5/popups/popup5-i-2.html?ou=693191) in main(). Note that the call to Thread.sleep() is now in the main() method, not the ActionListener for the JButton. If you press the button after the ten-second delay in main() is finished, the program works correctly. However, if you press the JButton before the ten-second delay is finished, the JButton keeps an accurate count for the first 10 seconds, then resets to 1 and starts over.

Figure 5.2



Button Program with Logic in both Main and Event Loop Threads

```
import java.awt.event.*;

public class GUIExample2 {
private static int counter = 0;

public static void main(String args[]) {
JFrame frame = new JFrame("Counter");
Container con = frame.getContentPane();
con.setLayout(new FlowLayout());
```

```
con.add(new JLabel("Counter"));
final JTextField counterText = new JTextField("0 ");
con.add(counterText);

JButton counterButton = new JButton("Increment");
counterButton.addActionListener(new ActionListener() {
public void actionPerformed(ActionEvent ae) {
counter++;
counterText.setText("" + counter);
}
});
con.add(counterButton);

JButton exitButton = new JButton("Exit");
exitButton.addActionListener(new ActionListener() {
public void actionPerformed(ActionEvent ae) {
System.exit(0);
}
});
con.add(exitButton);

frame.pack();
frame.setVisible(true);

// NOTE: Sleep is moved to main method, which puts
// it in the main thread, not the Event Loop
// which is running the JButton. Note that any
// increments done by the JButton after 10
// seconds are lost.
try {
Thread.sleep(10000);
counter = 1;
} catch (InterruptedException ie) {
```

```
}
}
}
```

What makes these two programs different? The difference comes down to whether the sleep occurs in the same thread of execution as the JButton counter (as in figure 5.1), or in a different thread of execution from the JButton counter (as in figure 5.2). The second program shows the use of threads in that the sleep occurs concurrently with the JButton being pressed. Thus, the second program has at least two threads: one for the main method where the sleep occurs, and one for handling the JButton.

As this second program shows, every program which uses the Java Swing or AWT classes is multithreaded and thus concurrent. If you have written at least one program using Swing or AWT, you have written a concurrent program. Therefore, the first reason to study concurrency is that it appears throughout Java programs. A programmer does not have to explicitly use Threads to have a concurrent program in Java; most concurrency in Java is implicit, and indeed hidden from a programmer.

There is also a second reason to study concurrent programming. As the second example illustrates, not realizing that a program is concurrent and not understanding the implications of concurrency can often result in program errors. These errors can be unpredictable, difficult to reproduce, and hard to fix. For example, consider a situation in which the counter application is used to keep track of the number of people entering a park. Most of the time, the system might function correctly, as people will typically not come to the park as soon as it opens. On certain days, however, the system might not work correctly, such as on a sunny Saturday when a crowd of people shows up at the gate at opening time. This relationship between the days of the week the program is run and the correctness of the program is complex and not obvious. This type of bug would be hard to identify, and, without understanding concurrent programming, difficult to fix.

Although the examples given show that concurrent programming is implicit in GUI programming, concurrent programming does not only occur when you are writing GUI programs using AWT or Swing. There are many other examples where threads are created implicitly, and are thus examples of concurrent programming in Java.

For example, because most Web servers spawn a new thread for each transaction they receive, most programs written on a server to process transactions are concurrent. Databases generally accept queries and updates from multiple sources, so any application that uses a database is concurrent. A program that implements or consumes services in a distributed environment (one that uses multiple computers) is concurrent. Finally, most computer chips today use hyperthreaded (multiple threads in a single cpu) or multicore (multiple CPUs on a single chip) architectures, and to utilize the power, these chips require that applications be written using concurrency.

Because concurrent programming is ubiquitous and can result in incorrect programs, it is important to know how concurrency works, the types of problems it can cause, and how to write correct concurrent programs.

B. Thread-based Programs

Although most concurrency in programs occurs through *hidden* threads created for the programmer (for example, the threads created when using Swing in the examples in the previous section), it is necessary to explicitly create and run threads in Java to understand threading. First, we will show you how to declare and execute java Thread objects. Then, we will show an example comparing a procedural program to a program with multiple threads. The next section of this module will show program errors that are unique to concurrent programs, and how to solve those errors. Finally, we will give a simple example of how to write objects for a concurrent environment using threads and state diagrams.

C. Creating Threads

There are two ways to create a thread in Java. The first involves creating an object that implements a Runnable interface, then passing it to a Thread object's constructor. The thread is then started, and execution begins in the *run()* method of the object passed to the thread. The second way is to extend the class Thread, and add a run() method. The Thread object is then started, and the execution begins in the run() method, as in the previous example. Both of these ways of creating a thread will be covered in the next sections.

Threads and Objects that Implement Runnable

The first, and probably the best, way to create a Thread in a program is to create an instance of an object that implements the Runnable interface. This is illustrated in the following program. In this program, a class MyRunnable is created, and it is given a run() method that simply prints out the message "In run method". When the main() thread starts, an instance of the class MyRunnable is created, and passed to the constructor for the Thread class, creating a Thread object named ct1. The start() method is now called on the Thread object, which in turn begins execution in the run() method of the MyRunnable object. The program then prints out "In run method". The thread ct1 finishes, the main thread finishes, and the program is completed.

```
public class MyRunnable implements Runnable {
public static void main(String[] args) {
MyRunnable ct1 = new MyRunnable();
Thread thread1 = new Thread(ct1);
thread1.start();
}

public void run() {
System.out.println("In run method");
}
}
```

Note the use of the term *begins execution in the run() method*. As will be shown in a following section, the run() in the MyRunnable object is actually executed in a separate thread from the main. It is incorrect to talk about the *start() method calling the run() method*. This implies that the two are run in the same thread (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m5-module-5/popups/popup5-i-3.html?ou=693191) , which is not the case. The run method in the MyRunnable object is executed in a different thread from the thread that is executing in the main method. While the result is the same in this simple program, the difference is important, and will be illustrated in later sections in this module.

In the above program, the identifier myRunnable is only created to be passed to the Thread constructor, so there really is no reason to create an identifier for it. Therefore, the instance of the MyRunnable class can be created directly in the constructor for the Thread class, as illustrated in the following program. These objects can be referred to as anonymous objects.

```
public class MyRunnable implements Runnable {
public static void main(String[] args) {
Thread thread1 = new Thread(new MyRunnable());
thread1.start();
}

public void run() {
System.out.println("In run method");
}
}
```

Following this logic one step further, the Thread identifier is only created so that the start() method can be called on this variable. Once again, there is no need to create an identifier to do this, as it can all be done in a single statement using two anonymous objects, as shown in the following program. The statement now creates the MyRunnable object, passes it to the constructor for the Thread class creating the Thread object, and then directly calls the start() method on this object without creating an identifier. While this may seem confusing at first, the single-statement with anonymous objects approach is much easier to use when writing programs, and will be used extensively throughout this course.

```
public class MyRunnable implements Runnable {
public static void main(String[] args) {
(new Thread(new MyRunnable())).start();
}

public void run() {
```

```
System.out.println("In run method");
}
}
```

Calling the start() method on a Thread object without creating an identifier may cause some confusion. Because there is no identifier, it appears that the Thread object is immediately out of scope, and so can be garbage collected. However, when a Thread object is created, it is registered with the JVM, so that a reference to this object exists even though there is no identifier in the program. Therefore, a Thread object cannot be garbage collected until the thread has completed execution.

Extending class Thread

The second way to create a Thread object is to extend class Thread. Since Thread implements the Runnable interface, and has a run() method defined, this is perfectly valid. And since all methods in Java are virtual, the run() method defined in the sub class will be executed when the start() method is called on a Thread, which is exactly what is expected.

```
public class MyThread extends Thread {
public static void main(String[] args) {
(new MyThread()).start();
}

public void run() {
System.out.println("In run method");
}
}
```

The method of creating an instance of a class that implements Runnable and passing it to the Thread constructor, as demonstrated in the previous section, is actually preferable to the method of creating a Thread object shown above. There are a number of reasons for this:

First, since Java has only single inheritance, extending the Thread class means that the class that is created cannot extend any other class. While there might be no plans to have this subclass ever extend another class, the question is, what is to be gained by extending class Thread instead of using a Runnable object? In this case, extending class Thread costs something, even though it may be small, for no gain.

Second, many libraries—for example, the ThreadPoolExecutor class in Java.util.concurrent—want to use Runnable objects. Although an object that extends Thread would work because Thread implements Runnable, the object would still generate all the extra overhead of the Thread class, which is a very high amount of overhead. This is a potentially large drag on the system.

Finally, a class that extends Thread is typically not a specialized type of Thread, but is related to some entity in the program. For example, you might have a bank with 25 Customer objects that you want to execute as separate threads. In this case, it is not really appropriate to say a Customer is-a Thread. The Customer object is-a Thread only in the limited sense of how it the object is implemented, not in what it means in the problem. Therefore, extending a thread violates the most basic rule of inheritance—namely, that inheritance should represent an "is-a" relationship.

So, unless there is some overarching reason to extend Thread, we recommend that you create a class that implements Runnable, and pass it to the Thread constructor, as illustrated in the previous section.

D. Running Threads

The actual execution of a thread is illustrated by contrasting it with a procedural program that you are probably more familiar with.

When the threads are active and can be executed, the virtual machine (VM) is free to choose any of the threads to execute. Thus, any thread is equally able to execute at any point in time. There are multiple execution paths through the program, with multiple outputs possible. Because each execution path is ordered, these outputs are not totally unordered. But the overall output is only *partially ordered*, as many different orderings are possible depending on how the threads are executed. Thus,

you cannot predict the output from the program. (See Self-Assessment program 2 for an example of how quickly ordered execution paths can result in a large number of different outputs).

E. ThreadPoolExecutor

Creating and starting a thread can cause significant overhead. Although this does not matter for threads that have lifetimes very similar to the lifetime of the program, some applications have threads that start and stop often during a program's lifetime. An example of this would be a Web server wherein each time a transaction is created it is run in a separate thread. Since a Web server runs literally thousands of transactions a second, it is often inefficient to start and stop a thread each time a transaction is run. For these types of systems, Java introduced the ThreadPoolExecutor class.

The ThreadPoolExecutor works just the same as the Thread class, except that it creates a pool of threads that it starts and does not stop. When a transaction needs a thread, it retrieves one from the ThreadPoolExecutor, and uses it as if it were a normal thread. When the transaction has finished using the thread, it simply returns the thread to the ThreadPoolExecutor to be reused.

The ThreadPoolExecutor is only applicable to large, complex systems, and is not useful for the programs created in this class. The important point we want to make is that a method exists to reduce the overhead inherent in using threads.

II. Threats in Concurrent Programming

As we showed in the GUI counter program, there are threats to proper program execution that exist in a concurrent program, but not in a procedural program. These threats arise from the relationships and the timing considerations among threads. There are two types of threats to the proper execution of a concurrent program. The first threat is to program safety. The question to ask regarding program safety is, "if a program finishes, is the answer correct?" The most common problem with program safety is called a *race condition*.

The second threat in concurrent programming is liveness. The question to ask when considering liveness is, "does the program ever finish?" The most common problem with program liveness is called a *deadlock condition*. This section will examine race conditions and deadlock conditions.

A. Race Conditions

Race conditions occur when two threads are contending for the same resource (such as a variable), and perform an operation on that resource that requires more than one machine instruction (https://leocontent.umgc.edu/content/umuc/tus/cmsc/cmsc335/2225/modules/m5-module-5/popups/popup5-i-4.html?ou=693191) . For example, in the following program, a swap operation is performed. In the first step of the swap operation, the thread stores a value in a temporary shared variable, which it plans to retrieve later. If at some point during the execution of that method another thread is able to enter that swap method and change the value, the swap will not give the correct answer.

Note that there is a *race* occurring in this program. If the first thread can complete execution of the swap before the second thread enters and changes the value of the shared variable, the first thread *wins the race*, and the program gives the correct answer. However, if the second thread is able to change the value before the first thread can complete the method, the first thread *loses the race*, and the program gives an incorrect answer.

Race conditions in programs are particularly vexing. They can happen unpredictably, which makes them very difficult to reproduce and find. A race condition might only occur every thousandth time a program is run, which is too frequent for a production application, but too infrequent to allow for easy debugging. Race conditions are also more likely to happen on production systems, where the system (CPU, IO, etc.) utilization is greater than on a lightly used test system, which can make them even more difficult to reproduce in a test environment. It is not unusual for it to take days, weeks, or months to find a race condition, but then take only hours to fix it.

The most basic way to fix a race condition is to lock a section of code before it is entered so that only one thread can run in a critical section at any time.

This type of solution is easy to implement, and handles many of the race conditions that are present in a program. When a system is not amenable to this way of solving a race condition, there are numerous other strategies to try. However, they are all beyond the scope of this class.

B. Deadlock

As mentioned above, deadlock is a situation in which two or more threads share two or more resources, and try to obtain them in such a way as to block the progress of both threads. In a deadlock situation, no thread can finish, and no thread can free up resources. The affected threads sit idle on the computer.

For deadlock to happen, four conditions must hold simultaneously. These are

1. Mutual exclusion. The resources must be nonsharable—that is, only one thread can use a resource at a time. This is true of the Locks class that was used in the previous example; in fact, this Locks class was used to ensure that only one thread could enter a critical section at a time.

2. Hold and wait. At least one of the threads must use two Locks, and must be able to hold one Lock while waiting to acquire the second Lock. This thread can block another thread attempting to acquire the first Lock, while being blocked itself from trying to obtain the second Lock.

3. No preemption. No thread can force any other thread to give up a Lock. A thread can only give up the Lock voluntarily. It typically does this by completing its task and calling unlock (later in this module, we will show how this can be done using Java Condition objects).

4. Circular wait. The threads must acquire the Lock objects in a circular fashion, whereby the first thread holds a Lock needed by a second thread, and the second thread holds a Lock needed by the first thread.

While the example here is for Lock objects, there are many other ways to cause deadlocks. Perhaps the most common deadlock situation in modern computing occurs in databases. It is not uncommon for a series of queries to lock records in the database so that there is a deadlock within the database. The normal solution to this problem is to kill all the queries that are involved in the deadlock, have them roll back the transaction, and start them again after a random interval.

III. Designing Objects for Concurrency

In the last section of the module, we will look at how to design objects to be used in concurrent programs. We provide an example that uses active objects (threads) and passive objects to implement a classic concurrent program known as a producer/consumer (or bounded buffer) problem. The example calls for the following steps:

1. Find the active and passive objects in the problem.
2. Write the active objects. Note that, in Java, active objects can be implemented as threads.
3. Draw a state diagram of the passive object.
4. Implement the state diagram version of the passive object.
5. Write a control object, which creates all the objects in the problem and starts any threads.

Note that this methodology will use Java Condition objects. In order to show you how to implement the code in this section, we will first discuss Condition objects.

A. Condition Variables

The Lock objects previously introduced work well for implementing critical sections. However, it is sometimes necessary to have a program release a Lock object until some specific condition is realized. The mechanism by which this occurs is called a Condition object.

A Condition object is an object that is associated with a Lock object. You can create a Condition object on a Lock object simply by calling the newCondition() method on the Lock object. This creates a Condition object for you (this is another example of a Factory pattern).

A Condition object works by temporarily releasing the associated lock on the Lock object and going into a wait state until the condition it is awaiting is fulfilled. When another thread calls the signal() method on this Condition, the condition is satisfied, and the thread that called the await() method on the Condition object is again ready to run. It must first, however, re-obtain the lock on the associated Lock object.

Be careful when using Condition objects with ReentrantLock objects. Each time you call a ReentrantLock object's lock() method, the call adds one to the number of unlock() calls that must be made before the lock is actually released. However, when you call the await() method on the Condition object, the lock is released, regardless of how many times you called the lock() method.
Thus await() and signal() can be unmatched, but you must match each call to lock() with a call to unlock(). Neglecting this step can cause strange errors if care is not taken, as will be shown in self-assessment question 8.

B. Producer/Consumer Problem

The Producer/Consumer problem involves two or more threads, one or more of which produces data in some fashion, and one or more of which consumes that data in some fashion. In the problem there will be one Producer thread and one Consumer thread. The Producer thread will create 10 integers from 0 to 9. The Consumer thread will take the 10 integers and print them out to the console.

To facilitate the passing of data between the two threads, there is an intermediate object, which is called a *BoundedBuffer*. This object is called a BoundedBuffer because it can store several values from the Producer, thus buffering this data for the Producer so that it can continue to work until the Consumer is ready to take data from the buffer. The BoundedBuffer is bounded

in that it has a limited size—in this case, it can hold two data values. Although the Producer can create more than two data values, it must wait to add data to a full BoundedBuffer until the Consumer has removed one value.

In the producer/consumer problem, the Producer will simply create integers from 1 to 10, and put them to in BoundedBuffer. The Consumer will run until it has taken all 10 values from the BoundedBuffer.

C. Finding the Objects

First, the active and passive objects in the problem are found. Objects in a problem can generally be found by looking for the nouns in the problem. From the description above, the objects needed are the BoundedBuffer, Producer, and Consumer.

In this problem, the Producer and Consumer are active in that they act on the BoundedBuffer, putting data into the BoundedBuffer and taking data from the BoundedBuffer.

The BoundedBuffer is passive in that it takes data from the Producer and gives data to the Consumer. It does nothing on its own; it only responds to requests from the active objects.

D. Writing the Active Objects

Now the active objects are written. When you write an active object, you must create a Thread object to run the object, and code the activities that are to be done by the object in the run() method. In this case, the run() method for the Producer consists of a loop that generates the integer values from 1 to numberOfItems, and adds them to the BoundedBuffer. The run method for the Consumer is a loop from 1 to numberOfItems, which takes the data from the BoundedBuffer and prints it to the console.

The loop itself is relatively easy to implement; however, the problem involves additional steps. It is necessary for the Producer to know how many items to produce, and to have the reference to the BoundedBuffer object that it is passing

data to. These data items cannot be passed into the run() method directly, as the run() method does not take any parameters. Instead, these values are set in the constructor when the object is created. The following is the code for the Producer object:

```
class Producer implements Runnable {
private BoundedBuffer _sb;
public Producer(BoundedBuffer sb) {
_sb = sb;
}

public void run() {
try {
for (int i = 0; i < 10; i++) {
Thread.sleep((int)(Math.random() * 100));
System.out.println("Adding item " + i);
_sb.put(i);
}
} catch(InterruptedException ie) {
}

System.out.println("Producer Finished");
}
}
```

The Consumer object is simply the mirror of the Producer object; it takes the data from the BoundedBuffer and consumes it by printing it out. The following is the code for the Consumer object:

```
class Consumer implements Runnable {
private BoundedBuffer _sb;
public Consumer(BoundedBuffer sb) {
_sb = sb;
}
```

```
public void run() {
try {
for (int i = 0; i < 10; i++) {
Thread.sleep((int)(Math.random() * 1000));
int value = _sb.take();
System.out.println("Got item " + value);
}
} catch(InterruptedException ie) {
}
System.out.println("Consumer Finished");
}
}
```

Now that the Producer and Consumer objects have been written, the next step is to write the passive BoundedBuffer object.

E. Designing the Passive Object

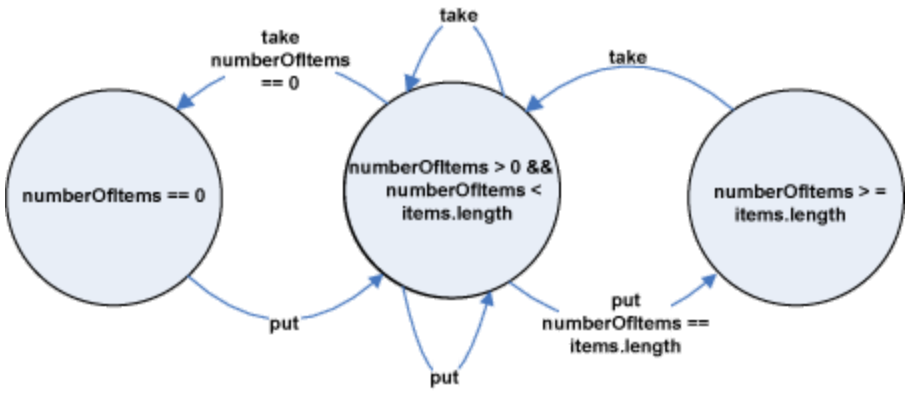
The passive object is more complex to write than the active objects because it does not have a thread of execution to determine what it will do next. Instead, it responds to requests from the active objects.

Passive objects are different than other objects in that you cannot always execute any method in the object. Because these objects are used with threads, the methods must lock and unlock critical sections. Also, the data in the object may preclude a method from running. For example, if the BoundedBuffer is currently full, it cannot process calls to its put() method, so the thread calling the put() method has to wait until a call to the take() method removes an item and makes space in the BoundedBuffer for the item.

To model the BoundedBuffer to only run a method when it is valid to do so, a State Diagram can be used. Figure 5.3 is an example of a State Diagram for the BoundedBuffer. In this diagram, circles represent states. There are three states in which the BoundedBuffer can exist: it can be needEmptySpace (it has no empty

space and cannot receive data); it can be needItem (it has no data, and no object can take data from it); and it can be partiallyFull (it has both data and empty space so that data can be added and taken).

Figure 5.3



State Diagram for the Bounded Buffer

Conditions:
needItem: numberOfItems == 0
needEmptySpace: numberOfItems >= items.length

Methods for the BoundedBuffer are represented as arcs between the circles. If an arc starts from a circle, the method can be run from that state. For example, the put() method can be run from either an needEmptySpace or partiallyFull state. The take() method can be run from either a needItem or partiallyFull state. Note that the needEmptySpace and partiallyFull states are true when the numberOfElements < elements.length(); and the needItem and partiallyFull states are true when the numberOfElements > 0. In the program, these states will be represented with the number of items in the queue.

Also note that Condition objects need to be created to allow the thread to wait for a specific condition. There must be one Condition object that specifies each Condition that a thread can wait on. In this example, the Condition objects needEmptySpace and needItem are created to allow waiting queues for threads which enter the put or take method but that cannot currently execute.

Now that the state diagram is completed, it can be translated into a Java object. In the next section, we show how this is done.

F. Creating the Passive Object

To create the passive object, you would simply translate the state diagram developed in the previous step into a Java class that can be instantiated to create an object. You can do this in four steps:

1. Create the basic outline of the object, using the arcs as the methods for the object. Note at this point that only the methods are declared; the instance variables and method definitions are completed in later steps.

2. Put a precondition into the method that allows the method only to be run when the precondition is true. This precondition represents the state or states that all arcs with the same name (that is, all arcs that are the same method) start from in the state diagram. Note that, to implement these preconditions, you would use Lock objects and Condition objects.

3. Add a postcondition to the method to represent the new state of the passive object. This postcondition represents the state at which the arc ends.

4. Finally, add any processing to implement the object to the method between the pre- and postcondition processing.

Implement the Passive Object Outline

The first step in converting a state diagram into a passive object is to provide the basic outline of the object. In this step, the class for the passive object is defined, as well as the method declarations (only) for this class. For the Producer/Consumer problem, the class is defined and the methods are declared in the class definition below.

Note that, in this method, we have included a Lock object, and that this Lock object is locked when the method is entered and unlocked when this method is exited. The inclusion of this Lock object is necessary because the methods in a passive object will operate on shared data; thus, the methods in a passive object need to be treated as critical sections of code.

Finally, note that the return statement must come after the finally block in this program. Note that this means that any data returned will not be in a critical section, so only local variables should be returned. It will not be safe to return instance or static variables as they will not be guaranteed to be correct because of race conditions.

```
class BoundedBuffer{
private ReentrantLock _bufferLock = new ReentrantLock();

public BoundedBuffer(int size) {
}

public int take() {
try {
_bufferLock.lock();
} catch (InterruptedException ie) {
} finally {
_bufferLock.unlock();

}
return null;
}

public void put(int item) {
try {
_bufferLock.lock();
} catch (InterruptedException ie) {
} finally {
_bufferLock.unlock();

}
}
}
```

Implement the Preconditions

The next step in implementing a passive object is to implement the preconditions the program must check to ensure that the condition to run the method is valid. These conditions are represented by the states in the state diagrams that the arcs (or methods) come from. If the condition to run the method is valid, the execution can continue. If not, the program must wait on a condition variable that represents the condition that must be valid before the method can execute. This is shown for the BoundedBuffer in the following class definition.

Note that the preconditions are checked in a while loop. This is to account for the following race condition: Remember that, when a call to the await() method is made, the underlying lock for the Condition object is dropped. The lock must be reacquired by the thread that has been made ready by the call to the signal() method. However, between the time the thread has been made ready and the time it is able to acquire the lock, another thread can call the lock method on the Lock object. It can thus change the passive object (in this case, the BoundedBuffer) so that the precondition is no longer valid. Thus, every precondition should use a while loop to check whether the precondition is still valid.

```
class BoundedBuffer{
private ReentrantLock _bufferLock = new ReentrantLock();
private Condition _needItem = _bufferLock.newCondition();
private Condition _needEmptySpace = _bufferLock.newCondition();

private int[] _items;
private int _numberOfItems;

public BoundedBuffer(int size) {
_items = new int[size];
}

public int take() {
try {
_bufferLock.lock();
while(_numberOfItems == 0) {
```

```
_needItem.await();
}
} catch (InterruptedException ie) {
} finally {
_bufferLock.unlock();
}
return null;
}

public void put(int item) {
try {
_bufferLock.lock();
while(_numberOfItems >= _items.length) {
_needEmptySpace.await();
}
} catch (InterruptedException ie) {
} finally {
_bufferLock.unlock();
}
}
}
```

Implement the Postconditions

The next step is to implement the postcondition processing. The postconditions are represented in the state diagrams and are pointed to by the arcs (or methods). If a program stays in the same state, you know that no thread currently waiting on a condition will be able to run, so there is no need to signal() any Condition objects. However, if the passive object changes state, the program should check to see whether there are any threads waiting on that Condition object. If there are, then a call to the signal() method on that Condition object should be made.

It is not strictly necessary to check to see whether there are threads waiting on the condition because, if no threads are waiting, the call is simply ignored. However, the call to signal could be expensive in some situations, so it is always a good idea

to check for waiting threads. In this case, it would likely incur little penalty to call signal()on the Condition object, but the check for waiting threads is added for illustration purposes.

The source code below shows the modifications to the BoundedBuffer class to add postcondition processing.

```
class BoundedBuffer{
private ReentrantLock _bufferLock = new ReentrantLock();
private Condition _needItem = _bufferLock.newCondition();
private Condition _needEmptySpace = _bufferLock.newCondition();

private int[] _items;
private int _firstItem;
private int _lastItem;
private int _numberOfItems;

public BoundedBuffer(int size) {
_items = new int[size];
}
public int take() {
int returnItem = 0;

try {
_bufferLock.lock();
while(_numberOfItems == 0) {
_needItem.await();
}

if (_bufferLock.hasWaiters(_needEmptySpace))
_needEmptySpace.signal();
} catch (InterruptedException ie) {
} finally {
_bufferLock.unlock();
```

```
}
return returnItem;
}

public void put(int item) {
try {
_bufferLock.lock();
while(_numberOfItems >= _items.length) {
_needEmptySpace.await();
}

if (_bufferLock.hasWaiters(_needItem))
_needItem.signal();
} catch (InterruptedException ie) {
} finally {
_bufferLock.unlock();
}
}
}
```

Implement any Processing Needed by the Method

The final step in implementing a passive object is to add any code needed to process the items between the preconditions and postconditions. In the BoundedBuffer class, this is the code needed to manage the buffer. This final, complete, BoundedBuffer class is given in the following source code:

```
class BoundedBuffer{
private ReentrantLock _bufferLock = new ReentrantLock();
private Condition _needItem = _bufferLock.newCondition();
private Condition _needEmptySpace = _bufferLock.newCondition();

private int[] _items;
private int _firstItem;
private int _lastItem;
private int _numberOfItems;
```

```
public BoundedBuffer(int size) {
_numberOfItems = 0;
_items = new int[size];
_firstItem = 0;
_lastItem = 0;
}

public int take() {
int returnItem = 0;

try {
_bufferLock.lock();
while(_numberOfItems == 0) {
_needItem.await();
}

returnItem = _items[_firstItem];
_firstItem = (++_firstItem % _items.length);
_numberOfItems--;

if (_bufferLock.hasWaiters(_needEmptySpace))
_needEmptySpace.signal();
} catch (InterruptedException ie) {
} finally {
_bufferLock.unlock();
}
return returnItem;
}

public void put(int item) {
try {
_bufferLock.lock();
while(_numberOfItems >= _items.length) {
_needEmptySpace.await();
```

```
}

_items[_lastItem] = item;
_lastItem = (++_lastItem % _items.length);
_numberOfItems++;

if (_bufferLock.hasWaiters(_needItem))
_needItem.signal();
} catch (InterruptedException ie) {
} finally {
_bufferLock.unlock();
}
}
}
```

G. Control Objects

Now that you have created all the active and passive objects, the final step is to put them together to generate a program. This is done by creating the passive object(s) needed to coordinate the active objects, then creating and starting the active objects. The following object accomplishes this goal for
the Producer/Consumer problem:

```
public class ProducerConsumer {
public static void main(String[] args) {
BoundedBuffer sb = new BoundedBuffer(2);
(new Thread(new Producer(sb))).start();
(new Thread(new Consumer(sb))).start();
}
}
```

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Synchronization

Threads communicate primarily by sharing access to fields and the objects reference fields refer to. This form of communication is extremely efficient, but makes two kinds of errors possible: *thread interference* and *memory consistency errors*. The tool needed to prevent these errors is *synchronization*.

However, synchronization can introduce *thread contention*, which occurs when two or more threads try to access the same resource simultaneously *and* cause the Java runtime to execute one or more threads more slowly, or even suspend their execution. Starvation and livelock are forms of thread contention. See the section Liveness for more information.

This section covers the following topics:

- Thread Interference describes how errors are introduced when multiple threads access shared data.
- Memory Consistency Errors describes errors that result from inconsistent views of shared memory.
- Synchronized Methods describes a simple idiom that can effectively prevent thread interference and memory consistency errors.
- Implicit Locks and Synchronization describes a more general synchronization idiom, and describes how synchronization is based on implicit locks.
- Atomic Access talks about the general idea of operations that can't be interfered with by other threads.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: The SimpleThreads Example
Next page: Thread Interference

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: Synchronization

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Thread Interference

Consider a simple class called `Counter`

```
class Counter {
    private int c = 0;

    public void increment() {
        c++;
    }

    public void decrement() {
        c--;
    }

    public int value() {
        return c;
    }

}
```

`Counter` is designed so that each invocation of `increment` will add 1 to `c`, and each invocation of `decrement` will subtract 1 from `c`. However, if a `Counter` object is referenced from multiple threads, interference between threads may prevent this from happening as expected.

Interference happens when two operations, running in different threads, but acting on the same data, *interleave*. This means that the two operations consist of multiple steps, and the sequences of steps overlap.

It might not seem possible for operations on instances of `Counter` to interleave, since both operations on `c` are single, simple statements. However, even simple statements can translate to multiple steps by the virtual machine. We won't examine the specific steps the virtual machine takes — it is enough to know that the single expression `c++` can be decomposed into three steps:

1. Retrieve the current value of `c`.
2. Increment the retrieved value by 1.
3. Store the incremented value back in `c`.

The expression `c--` can be decomposed the same way, except that the second step decrements instead of increments.

Suppose Thread A invokes `increment` at about the same time Thread B invokes `decrement`. If the initial value of `c` is `0`, their interleaved actions might follow this sequence:

1. Thread A: Retrieve c.
2. Thread B: Retrieve c.
3. Thread A: Increment retrieved value; result is 1.
4. Thread B: Decrement retrieved value; result is -1.

| Ad Choices

5. Thread A: Store result in c; c is now 1.
6. Thread B: Store result in c; c is now -1.

Thread A's result is lost, overwritten by Thread B. This particular interleaving is only one possibility. Under different circumstances it might be Thread B's result that gets lost, or there could be no error at all. Because they are unpredictable, thread interference bugs can be difficult to detect and fix.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Synchronization
Next page: Memory Consistency Errors

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: Synchronization

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Memory Consistency Errors

Memory consistency errors occur when different threads have inconsistent views of what should be the same data. The causes of memory consistency errors are complex and beyond the scope of this tutorial. Fortunately, the programmer does not need a detailed understanding of these causes. All that is needed is a strategy for avoiding them.

The key to avoiding memory consistency errors is understanding the *happens-before* relationship. This relationship is simply a guarantee that memory writes by one specific statement are visible to another specific statement. To see this, consider the following example. Suppose a simple `int` field is defined and initialized:

```
int counter = 0;
```

The `counter` field is shared between two threads, A and B. Suppose thread A increments `counter`:

```
counter++;
```

Then, shortly afterwards, thread B prints out `counter`:

```
System.out.println(counter);
```

If the two statements had been executed in the same thread, it would be safe to assume that the value printed out would be "1". But if the two statements are executed in separate threads, the value printed out might well be "0", because there's no guarantee that thread A's change to `counter` will be visible to thread B — unless the programmer has established a happens-before relationship between these two statements.

There are several actions that create happens-before relationships. One of them is synchronization, as we will see in the following sections.

We've already seen two actions that create happens-before relationships.

- When a statement invokes `Thread.start`, every statement that has a happens-before relationship with that statement also has a happens-before relationship with every statement executed by the new thread. The effects of the code that led up to the creation of the new thread are visible to the new thread.
- When a thread terminates and causes a `Thread.join` in another thread to return, then all the statements executed by the terminated thread have a happens-before relationship with all the statements following the successful join. The effects of the code in the thread are now visible to the thread that performed the join.

For a list of actions that create happens-before relationships, refer to the Summary page of the `java.util.concurrent` package..

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Thread Interference
Next page: Synchronized Methods

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: Synchronization

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Synchronized Methods

The Java programming language provides two basic synchronization idioms: *synchronized methods* and *synchronized statements*. The more complex of the two, synchronized statements, are described in the next section. This section is about synchronized methods.

To make a method synchronized, simply add the `synchronized` keyword to its declaration:

```
public class SynchronizedCounter {
    private int c = 0;

    public synchronized void increment() {
        c++;
    }

    public synchronized void decrement() {
        c--;
    }

    public synchronized int value() {
        return c;
    }
}
```

If `count` is an instance of `SynchronizedCounter`, then making these methods synchronized has two effects:

- First, it is not possible for two invocations of synchronized methods on the same object to interleave. When one thread is executing a synchronized method for an object, all other threads that invoke synchronized methods for the same object block (suspend execution) until the first thread is done with the object.
- Second, when a synchronized method exits, it automatically establishes a happens-before relationship with *any subsequent invocation* of a synchronized method for the same object. This guarantees that changes to the state of the object are visible to all threads.

Note that constructors cannot be synchronized — using the `synchronized` keyword with a constructor is a syntax error. Synchronizing constructors doesn't make sense, because only the thread that creates an object should have access to it while it is being constructed.

Warning: When constructing an object that will be shared between threads, be very careful that a reference to the object does not "leak" prematurely. For example, suppose you want to maintain a `List` called `instances` containing every instance of class. You might be tempted to add the following line to your constructor:

```
instances.add(this);
```

But then other threads can use `instances` to access the object before construction of the object is complete.

Synchronized methods enable a simple strategy for preventing thread interference and memory consistency errors: if an object is visible to more than one thread, all reads or writes to that object's variables are done through `synchronized` methods. (An important exception: `final` fields, which

| Ad Choices

cannot be modified after the object is constructed, can be safely read through non-synchronized methods, once the object is constructed) This strategy is effective, but can present problems with liveness, as we'll see later in this lesson.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Memory Consistency Errors
Next page: Intrinsic Locks and Synchronization

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: Synchronization

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Intrinsic Locks and Synchronization

Synchronization is built around an internal entity known as the *intrinsic lock* or *monitor lock*. (The API specification often refers to this entity simply as a "monitor.") Intrinsic locks play a role in both aspects of synchronization: enforcing exclusive access to an object's state and establishing happens-before relationships that are essential to visibility.

Every object has an intrinsic lock associated with it. By convention, a thread that needs exclusive and consistent access to an object's fields has to *acquire* the object's intrinsic lock before accessing them, and then *release* the intrinsic lock when it's done with them. A thread is said to *own* the intrinsic lock between the time it has acquired the lock and released the lock. As long as a thread owns an intrinsic lock, no other thread can acquire the same lock. The other thread will block when it attempts to acquire the lock.

When a thread releases an intrinsic lock, a happens-before relationship is established between that action and any subsequent acquisition of the same lock.

Locks In Synchronized Methods

When a thread invokes a synchronized method, it automatically acquires the intrinsic lock for that method's object and releases it when the method returns. The lock release occurs even if the return was caused by an uncaught exception.

You might wonder what happens when a static synchronized method is invoked, since a static method is associated with a class, not an object. In this case, the thread acquires the intrinsic lock for the `Class` object associated with the class. Thus access to class's static fields is controlled by a lock that's distinct from the lock for any instance of the class.

Synchronized Statements

Another way to create synchronized code is with *synchronized statements*. Unlike synchronized methods, synchronized statements must specify the object that provides the intrinsic lock:

```
public void addName(String name) {
    synchronized(this) {
        lastName = name;
        nameCount++;
    }
    nameList.add(name);
}
```

In this example, the `addName` method needs to synchronize changes to `lastName` and `nameCount`, but also needs to avoid synchronizing invocations of other objects' methods. (Invoking other objects' methods from synchronized code can create problems that are described in the section on Liveness.) Without synchronized statements, there would have to be a separate, unsynchronized method for the sole purpose of invoking `nameList.add`.

Synchronized statements are also useful for improving concurrency with fine-grained synchronization. Suppose, for example, class `MsLunch` has two instance fields, `c1` and `c2`, that are never used together. All updates of these fields must be synchronized, but there's no reason to prevent an update of c1 from being interleaved with an update of c2 — and doing so reduces concurrency by creating unnecessary blocking. Instead of using

| Ad Choices

```
public class MsLunch {
    private long c1 = 0;
    private long c2 = 0;
    private Object lock1 = new Object();
    private Object lock2 = new Object();

    public void inc1() {
        synchronized(lock1) {
            c1++;
        }
    }

    public void inc2() {
        synchronized(lock2) {
            c2++;
        }
    }
}
```

Use this idiom with extreme care. You must be absolutely sure that it really is safe to interleave access of the affected fields.

Reentrant Synchronization

Recall that a thread cannot acquire a lock owned by another thread. But a thread *can* acquire a lock that it already owns. Allowing a thread to acquire the same lock more than once enables *reentrant synchronization*. This describes a situation where synchronized code, directly or indirectly, invokes a method that also contains synchronized code, and both sets of code use the same lock. Without reentrant synchronization, synchronized code would have to take many additional precautions to avoid having a thread cause itself to block.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Synchronized Methods
Next page: Atomic Access

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: Synchronization

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Atomic Access

In programming, an *atomic* action is one that effectively happens all at once. An atomic action cannot stop in the middle: it either happens completely, or it doesn't happen at all. No side effects of an atomic action are visible until the action is complete.

We have already seen that an increment expression, such as `c++`, does not describe an atomic action. Even very simple expressions can define complex actions that can decompose into other actions. However, there are actions you can specify that are atomic:

- Reads and writes are atomic for reference variables and for most primitive variables (all types except `long` and `double`).
- Reads and writes are atomic for *all* variables declared `volatile` (*including* `long` and `double` variables).

Atomic actions cannot be interleaved, so they can be used without fear of thread interference. However, this does not eliminate all need to synchronize atomic actions, because memory consistency errors are still possible. Using `volatile` variables reduces the risk of memory consistency errors, because any write to a `volatile` variable establishes a happens-before relationship with subsequent reads of that same variable. This means that changes to a `volatile` variable are always visible to other threads. What's more, it also means that when a thread reads a `volatile` variable, it sees not just the latest change to the `volatile`, but also the side effects of the code that led up the change.

Using simple atomic variable access is more efficient than accessing these variables through synchronized code, but requires more care by the programmer to avoid memory consistency errors. Whether the extra effort is worthwhile depends on the size and complexity of the application.

Some of the classes in the `java.util.concurrent` package provide atomic methods that do not rely on synchronization. We'll discuss them in the section on High Level Concurrency Objects.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Intrinsic Locks and Synchronization
Next page: Liveness

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Liveness

A concurrent application's ability to execute in a timely manner is known as its *liveness*. This section describes the most common kind of liveness problem, deadlock, and goes on to briefly describe two other liveness problems, starvation and livelock.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Atomic Access
Next page: Deadlock

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: Liveness

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Deadlock

Deadlock describes a situation where two or more threads are blocked forever, waiting for each other. Here's an example.

Alphonse and Gaston are friends, and great believers in courtesy. A strict rule of courtesy is that when you bow to a friend, you must remain bowed until your friend has a chance to return the bow. Unfortunately, this rule does not account for the possibility that two friends might bow to each other at the same time. This example application, `Deadlock`, models this possibility:

```
public class Deadlock {
    static class Friend {
        private final String name;
        public Friend(String name) {
            this.name = name;
        }
        public String getName() {
            return this.name;
        }
        public synchronized void bow(Friend bower) {
            System.out.format("%s: %s"
                + "  has bowed to me!%n",
                this.name, bower.getName());
            bower.bowBack(this);
        }
        public synchronized void bowBack(Friend bower) {
            System.out.format("%s: %s"
                + " has bowed back to me!%n",
                this.name, bower.getName());
        }
    }

    public static void main(String[] args) {
        final Friend alphonse =
            new Friend("Alphonse");
        final Friend gaston =
            new Friend("Gaston");
        new Thread(new Runnable() {
            public void run() { alphonse.bow(gaston); }
        }).start();
        new Thread(new Runnable() {
            public void run() { gaston.bow(alphonse); }
        }).start();
    }
}
```

| Ad Choices

When `Deadlock` runs, it's extremely likely that both threads will block when they attempt to invoke `bowBack`. Neither block will ever end, because each thread is waiting for the other to exit `bow`.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Liveness
Next page: Starvation and Livelock

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: Liveness

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Starvation and Livelock

Starvation and livelock are much less common a problem than deadlock, but are still problems that every designer of concurrent software is likely to encounter.

Starvation

Starvation describes a situation where a thread is unable to gain regular access to shared resources and is unable to make progress. This happens when shared resources are made unavailable for long periods by "greedy" threads. For example, suppose an object provides a synchronized method that often takes a long time to return. If one thread invokes this method frequently, other threads that also need frequent synchronized access to the same object will often be blocked.

Livelock

A thread often acts in response to the action of another thread. If the other thread's action is also a response to the action of another thread, then *livelock* may result. As with deadlock, livelocked threads are unable to make further progress. However, the threads are not blocked — they are simply too busy responding to each other to resume work. This is comparable to two people attempting to pass each other in a corridor: Alphonse moves to his left to let Gaston pass, while Gaston moves to his right to let Alphonse pass. Seeing that they are still blocking each other, Alphone moves to his right, while Gaston moves to his left. They're still blocking each other, so...

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Deadlock
Next page: Guarded Blocks

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Guarded Blocks

Threads often have to coordinate their actions. The most common coordination idiom is the *guarded block*. Such a block begins by polling a condition that must be true before the block can proceed. There are a number of steps to follow in order to do this correctly.

Suppose, for example `guardedJoy` is a method that must not proceed until a shared variable `joy` has been set by another thread. Such a method could, in theory, simply loop until the condition is satisfied, but that loop is wasteful, since it executes continuously while waiting.

```
public void guardedJoy() {
    // Simple loop guard. Wastes
    // processor time. Don't do this!
    while(!joy) {}
    System.out.println("Joy has been achieved!");
}
```

A more efficient guard invokes `Object.wait` to suspend the current thread. The invocation of `wait` does not return until another thread has issued a notification that some special event may have occurred — though not necessarily the event this thread is waiting for:

```
public synchronized void guardedJoy() {
    // This guard only loops once for each special event, which may not
    // be the event we're waiting for.
    while(!joy) {
        try {
            wait();
        } catch (InterruptedException e) {}
    }
    System.out.println("Joy and efficiency have been achieved!");
}
```

Note: Always invoke `wait` inside a loop that tests for the condition being waited for. Don't assume that the interrupt was for the particular condition you were waiting for, or that the condition is still true.

Like many methods that suspend execution, `wait` can throw `InterruptedException`. In this example, we can just ignore that exception — we only care about the value of `joy`.

Why is this version of `guardedJoy` synchronized? Suppose `d` is the object we're using to invoke `wait`. When a thread invokes `d.wait`, it must own the intrinsic lock for `d` — otherwise an error is thrown. Invoking `wait` inside a synchronized method is a simple way to acquire the intrinsic lock.

When `wait` is invoked, the thread releases the lock and suspends execution. At some future time, another thread will acquire the same lock and invoke `Object.notifyAll`, informing all threads waiting on that lock that something important has happened:

```
public synchronized notifyJoy() {
    joy = true;
    notifyAll();
}
```

| Ad Choices

Some time after the second thread has released the lock, the first thread reacquires the lock and resumes by returning from the invocation of `wait`.

Note: There is a second notification method, `notify`, which wakes up a single thread. Because `notify` doesn't allow you to specify the thread that is woken up, it is useful only in massively parallel applications — that is, programs with a large number of threads, all doing similar chores. In such an application, you don't care which thread gets woken up.

Let's use guarded blocks to create a *Producer-Consumer* application. This kind of application shares data between two threads: the *producer*, that creates the data, and the *consumer*, that does something with it. The two threads communicate using a shared object. Coordination is essential: the consumer thread must not attempt to retrieve the data before the producer thread has delivered it, and the producer thread must not attempt to deliver new data if the consumer hasn't retrieved the old data.

In this example, the data is a series of text messages, which are shared through an object of type `Drop`:

```
public class Drop {
    // Message sent from producer
    // to consumer.
    private String message;
    // True if consumer should wait
    // for producer to send message,
    // false if producer should wait for
    // consumer to retrieve message.
    private boolean empty = true;

    public synchronized String take() {
        // Wait until message is
        // available.
        while (empty) {
            try {
                wait();
            } catch (InterruptedException e) {}
        }
        // Toggle status.
        empty = true;
        // Notify producer that
        // status has changed.
        notifyAll();
        return message;
    }

    public synchronized void put(String message) {
        // Wait until message has
        // been retrieved.
        while (!empty) {
            try {
                wait();
            } catch (InterruptedException e) {}
        }
        // Toggle status.
        empty = false;
        // Store message.
        this.message = message;
        // Notify consumer that status
        // has changed.
        notifyAll();
    }
}
```

The producer thread, defined in `Producer`, sends a series of familiar messages. The string "DONE" indicates that all messages have been sent. To simulate the unpredictable nature of real-world applications, the producer thread pauses for random intervals between messages.

```
import java.util.Random;

public class Producer implements Runnable {
    private Drop drop;
```

| Ad Choices

```
    public Producer(Drop drop) {
        this.drop = drop;
    }

    public void run() {
        String importantInfo[] = {
            "Mares eat oats",
            "Does eat oats",
            "Little lambs eat ivy",
            "A kid will eat ivy too"
        };
        Random random = new Random();

        for (int i = 0;
             i < importantInfo.length;
             i++) {
            drop.put(importantInfo[i]);
            try {
                Thread.sleep(random.nextInt(5000));
            } catch (InterruptedException e) {}
        }
        drop.put("DONE");
    }
}
```

The consumer thread, defined in `Consumer`, simply retrieves the messages and prints them out, until it retrieves the "DONE" string. This thread also pauses for random intervals.

```
import java.util.Random;

public class Consumer implements Runnable {
    private Drop drop;

    public Consumer(Drop drop) {
        this.drop = drop;
    }

    public void run() {
        Random random = new Random();
        for (String message = drop.take();
             ! message.equals("DONE");
             message = drop.take()) {
            System.out.format("MESSAGE RECEIVED: %s%n", message);
            try {
                Thread.sleep(random.nextInt(5000));
            } catch (InterruptedException e) {}
        }
    }
}
```

Finally, here is the main thread, defined in `ProducerConsumerExample`, that launches the producer and consumer threads.

```
public class ProducerConsumerExample {
    public static void main(String[] args) {
        Drop drop = new Drop();
        (new Thread(new Producer(drop))).start();
        (new Thread(new Consumer(drop))).start();
    }
}
```

Note: The `Drop` class was written in order to demonstrate guarded blocks. To avoid re-inventing the wheel, examine the existing data structures in the Java Collections Framework before trying to code your own data-sharing objects. For more information, refer to the Questions and Exercises section.

| Ad Choices

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Starvation and Livelock
Next page: Immutable Objects

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Immutable Objects

An object is considered *immutable* if its state cannot change after it is constructed. Maximum reliance on immutable objects is widely accepted as a sound strategy for creating simple, reliable code.

Immutable objects are particularly useful in concurrent applications. Since they cannot change state, they cannot be corrupted by thread interference or observed in an inconsistent state.

Programmers are often reluctant to employ immutable objects, because they worry about the cost of creating a new object as opposed to updating an object in place. The impact of object creation is often overestimated, and can be offset by some of the efficiencies associated with immutable objects. These include decreased overhead due to garbage collection, and the elimination of code needed to protect mutable objects from corruption.

The following subsections take a class whose instances are mutable and derives a class with immutable instances from it. In so doing, they give general rules for this kind of conversion and demonstrate some of the advantages of immutable objects.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Guarded Blocks
Next page: A Synchronized Class Example

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: Immutable Objects

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

A Synchronized Class Example

The class, `SynchronizedRGB`, defines objects that represent colors. Each object represents the color as three integers that stand for primary color values and a string that gives the name of the color.

```
public class SynchronizedRGB {

    // Values must be between 0 and 255.
    private int red;
    private int green;
    private int blue;
    private String name;

    private void check(int red,
                       int green,
                       int blue) {
        if (red < 0 || red > 255
            || green < 0 || green > 255
            || blue < 0 || blue > 255) {
            throw new IllegalArgumentException();
        }
    }

    public SynchronizedRGB(int red,
                           int green,
                           int blue,
                           String name) {
        check(red, green, blue);
        this.red = red;
        this.green = green;
        this.blue = blue;
        this.name = name;
    }

    public void set(int red,
                    int green,
                    int blue,
                    String name) {
        check(red, green, blue);
        synchronized (this) {
            this.red = red;
            this.green = green;
            this.blue = blue;
            this.name = name;
```

| Ad Choices

```
            }
        }

        public synchronized int getRGB() {
            return ((red << 16) | (green << 8) | blue);
        }

        public synchronized String getName() {
            return name;
        }

        public synchronized void invert() {
            red = 255 - red;
            green = 255 - green;
            blue = 255 - blue;
            name = "Inverse of " + name;
        }
    }
```

`SynchronizedRGB` must be used carefully to avoid being seen in an inconsistent state. Suppose, for example, a thread executes the following code:

```
SynchronizedRGB color =
    new SynchronizedRGB(0, 0, 0, "Pitch Black");
...
int myColorInt = color.getRGB();      //Statement 1
String myColorName = color.getName(); //Statement 2
```

If another thread invokes `color.set` after Statement 1 but before Statement 2, the value of `myColorInt` won't match the value of `myColorName`. To avoid this outcome, the two statements must be bound together:

```
synchronized (color) {
    int myColorInt = color.getRGB();
    String myColorName = color.getName();
}
```

This kind of inconsistency is only possible for mutable objects — it will not be an issue for the immutable version of `SynchronizedRGB`.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Immutable Objects
Next page: A Strategy for Defining Immutable Objects

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: Immutable Objects

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

A Strategy for Defining Immutable Objects

The following rules define a simple strategy for creating immutable objects. Not all classes documented as "immutable" follow these rules. This does not necessarily mean the creators of these classes were sloppy — they may have good reason for believing that instances of their classes never change after construction. However, such strategies require sophisticated analysis and are not for beginners.

1. Don't provide "setter" methods — methods that modify fields or objects referred to by fields.
2. Make all fields `final` and `private`.
3. Don't allow subclasses to override methods. The simplest way to do this is to declare the class as `final`. A more sophisticated approach is to make the constructor `private` and construct instances in factory methods.
4. If the instance fields include references to mutable objects, don't allow those objects to be changed:
    - Don't provide methods that modify the mutable objects.
    - Don't share references to the mutable objects. Never store references to external, mutable objects passed to the constructor; if necessary, create copies, and store references to the copies. Similarly, create copies of your internal mutable objects when necessary to avoid returning the originals in your methods.

Applying this strategy to `SynchronizedRGB` results in the following steps:

1. There are two setter methods in this class. The first one, `set`, arbitrarily transforms the object, and has no place in an immutable version of the class. The second one, `invert`, can be adapted by having it create a new object instead of modifying the existing one.
2. All fields are already `private`; they are further qualified as `final`.
3. The class itself is declared `final`.
4. Only one field refers to an object, and that object is itself immutable. Therefore, no safeguards against changing the state of "contained" mutable objects are necessary.

After these changes, we have `ImmutableRGB`:

```
final public class ImmutableRGB {

    // Values must be between 0 and 255.
    final private int red;
    final private int green;
    final private int blue;
    final private String name;

    private void check(int red,
                       int green,
                       int blue) {
        if (red < 0 || red > 255
            || green < 0 || green > 255
            || blue < 0 || blue > 255) {
            throw new IllegalArgumentException();
        }
```

| Ad Choices

```
        }

        public ImmutableRGB(int red,
                            int green,
                            int blue,
                            String name) {
            check(red, green, blue);
            this.red = red;
            this.green = green;
            this.blue = blue;
            this.name = name;
        }


        public int getRGB() {
            return ((red << 16) | (green << 8) | blue);
        }

        public String getName() {
            return name;
        }

        public ImmutableRGB invert() {
            return new ImmutableRGB(255 - red,
                            255 - green,
                            255 - blue,
                            "Inverse of " + name);
        }
    }
```

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: A Synchronized Class Example
Next page: High Level Concurrency Objects

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

High Level Concurrency Objects

So far, this lesson has focused on the low-level APIs that have been part of the Java platform from the very beginning. These APIs are adequate for very basic tasks, but higher-level building blocks are needed for more advanced tasks. This is especially true for massively concurrent applications that fully exploit today's multiprocessor and multi-core systems.

In this section we'll look at some of the high-level concurrency features introduced with version 5.0 of the Java platform. Most of these features are implemented in the new `java.util.concurrent` packages. There are also new concurrent data structures in the Java Collections Framework.

- Lock objects support locking idioms that simplify many concurrent applications.
- Executors define a high-level API for launching and managing threads. Executor implementations provided by `java.util.concurrent` provide thread pool management suitable for large-scale applications.
- Concurrent collections make it easier to manage large collections of data, and can greatly reduce the need for synchronization.
- Atomic variables have features that minimize synchronization and help avoid memory consistency errors.
- `ThreadLocalRandom` (in JDK 7) provides efficient generation of pseudo-random numbers from multiple threads.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: A Strategy for Defining Immutable Objects
Next page: Lock Objects

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: High Level Concurrency Objects

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Lock Objects

Synchronized code relies on a simple kind of reentrant lock. This kind of lock is easy to use, but has many limitations. More sophisticated locking idioms are supported by the `java.util.concurrent.locks` package. We won't examine this package in detail, but instead will focus on its most basic interface, `Lock`.

`Lock` objects work very much like the implicit locks used by synchronized code. As with implicit locks, only one thread can own a `Lock` object at a time. `Lock` objects also support a `wait/notify` mechanism, through their associated `Condition` objects.

The biggest advantage of `Lock` objects over implicit locks is their ability to back out of an attempt to acquire a lock. The `tryLock` method backs out if the lock is not available immediately or before a timeout expires (if specified). The `lockInterruptibly` method backs out if another thread sends an interrupt before the lock is acquired.

Let's use `Lock` objects to solve the deadlock problem we saw in Liveness. Alphonse and Gaston have trained themselves to notice when a friend is about to bow. We model this improvement by requiring that our `Friend` objects must acquire locks for *both* participants before proceeding with the bow. Here is the source code for the improved model, `Safelock`. To demonstrate the versatility of this idiom, we assume that Alphonse and Gaston are so infatuated with their newfound ability to bow safely that they can't stop bowing to each other:

```
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;
import java.util.Random;

public class Safelock {
    static class Friend {
        private final String name;
        private final Lock lock = new ReentrantLock();

        public Friend(String name) {
            this.name = name;
        }

        public String getName() {
            return this.name;
        }

        public boolean impendingBow(Friend bower) {
            Boolean myLock = false;
            Boolean yourLock = false;
            try {
                myLock = lock.tryLock();
                yourLock = bower.lock.tryLock();
            } finally {
                if (! (myLock && yourLock)) {
```

| Ad Choices

```
                    lock.unlock();
                }
                if (yourLock) {
                    bower.lock.unlock();
                }
            }
        }
        return myLock && yourLock;
    }

    public void bow(Friend bower) {
        if (impendingBow(bower)) {
            try {
                System.out.format("%s: %s has"
                    + " bowed to me!%n",
                    this.name, bower.getName());
                bower.bowBack(this);
            } finally {
                lock.unlock();
                bower.lock.unlock();
            }
        } else {
            System.out.format("%s: %s started"
                + " to bow to me, but saw that"
                + " I was already bowing to"
                + " him.%n",
                this.name, bower.getName());
        }
    }

    public void bowBack(Friend bower) {
        System.out.format("%s: %s has" +
            " bowed back to me!%n",
            this.name, bower.getName());
    }
}

static class BowLoop implements Runnable {
    private Friend bower;
    private Friend bowee;

    public BowLoop(Friend bower, Friend bowee) {
        this.bower = bower;
        this.bowee = bowee;
    }

    public void run() {
        Random random = new Random();
        for (;;) {
            try {
                Thread.sleep(random.nextInt(10));
            } catch (InterruptedException e) {}
            bowee.bow(bower);
        }
    }
}


public static void main(String[] args) {
    final Friend alphonse =
        new Friend("Alphonse");
    final Friend gaston =
        new Friend("Gaston");
    new Thread(new BowLoop(alphonse, gaston)).start();
    new Thread(new BowLoop(gaston, alphonse)).start();
```

| Ad Choices

```
        }
    }
```

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: High Level Concurrency Objects
Next page: Executors

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: High Level Concurrency Objects

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Executors

In all of the previous examples, there's a close connection between the task being done by a new thread, as defined by its `Runnable` object, and the thread itself, as defined by a `Thread` object. This works well for small applications, but in large-scale applications, it makes sense to separate thread management and creation from the rest of the application. Objects that encapsulate these functions are known as *executors*. The following subsections describe executors in detail.

- Executor Interfaces define the three executor object types.
- Thread Pools are the most common kind of executor implementation.
- Fork/Join is a framework (new in JDK 7) for taking advantage of multiple processors.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Lock Objects
Next page: Executor Interfaces

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: High Level Concurrency Objects
Subsection: Executors

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Executor Interfaces

The `java.util.concurrent` package defines three executor interfaces:

- `Executor`, a simple interface that supports launching new tasks.
- `ExecutorService`, a subinterface of `Executor`, which adds features that help manage the life cycle, both of the individual tasks and of the executor itself.
- `ScheduledExecutorService`, a subinterface of `ExecutorService`, supports future and/or periodic execution of tasks.

Typically, variables that refer to executor objects are declared as one of these three interface types, not with an executor class type.

The `Executor` Interface

The `Executor` interface provides a single method, `execute`, designed to be a drop-in replacement for a common thread-creation idiom. If `r` is a `Runnable` object, and `e` is an `Executor` object you can replace

```
(new Thread(r)).start();
```

with

```
e.execute(r);
```

However, the definition of `execute` is less specific. The low-level idiom creates a new thread and launches it immediately. Depending on the `Executor` implementation, `execute` may do the same thing, but is more likely to use an existing worker thread to run `r`, or to place `r` in a queue to wait for a worker thread to become available. (We'll describe worker threads in the section on Thread Pools.)

The executor implementations in `java.util.concurrent` are designed to make full use of the more advanced `ExecutorService` and `ScheduledExecutorService` interfaces, although they also work with the base `Executor` interface.

The `ExecutorService` Interface

The `ExecutorService` interface supplements `execute` with a similar, but more versatile `submit` method. Like `execute`, `submit` accepts `Runnable` objects, but also accepts `Callable` objects, which allow the task to return a value. The `submit` method returns a `Future` object, which is used to retrieve the `Callable` return value and to manage the status of both `Callable` and `Runnable` tasks.

`ExecutorService` also provides methods for submitting large collections of `Callable` objects. Finally, `ExecutorService` provides a number of methods for managing the shutdown of the executor. To support immediate shutdown, tasks should handle interrupts correctly.

The `ScheduledExecutorService` Interface

The `ScheduledExecutorService` interface supplements the methods of its parent `ExecutorService` with `schedule`, which executes a `Runnable` or `Callable` task after a specified delay. In addition, the interface defines `scheduleAtFixedRate` and `scheduleWithFixedDelay`, which executes specified tasks repeatedly, at defined intervals.

| Ad Choices

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Executors
Next page: Thread Pools

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: High Level Concurrency Objects
Subsection: Executors

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Thread Pools

Most of the executor implementations in `java.util.concurrent` use *thread pools*, which consist of *worker threads*. This kind of thread exists separately from the `Runnable` and `Callable` tasks it executes and is often used to execute multiple tasks.

Using worker threads minimizes the overhead due to thread creation. Thread objects use a significant amount of memory, and in a large-scale application, allocating and deallocating many thread objects creates a significant memory management overhead.

One common type of thread pool is the *fixed thread pool*. This type of pool always has a specified number of threads running; if a thread is somehow terminated while it is still in use, it is automatically replaced with a new thread. Tasks are submitted to the pool via an internal queue, which holds extra tasks whenever there are more active tasks than threads.

An important advantage of the fixed thread pool is that applications using it *degrade gracefully*. To understand this, consider a web server application where each HTTP request is handled by a separate thread. If the application simply creates a new thread for every new HTTP request, and the system receives more requests than it can handle immediately, the application will suddenly stop responding to *all* requests when the overhead of all those threads exceed the capacity of the system. With a limit on the number of the threads that can be created, the application will not be servicing HTTP requests as quickly as they come in, but it will be servicing them as quickly as the system can sustain.

A simple way to create an executor that uses a fixed thread pool is to invoke the `newFixedThreadPool` factory method in `java.util.concurrent.Executors` This class also provides the following factory methods:

- The `newCachedThreadPool` method creates an executor with an expandable thread pool. This executor is suitable for applications that launch many short-lived tasks.
- The `newSingleThreadExecutor` method creates an executor that executes a single task at a time.
- Several factory methods are `ScheduledExecutorService` versions of the above executors.

If none of the executors provided by the above factory methods meet your needs, constructing instances of `java.util.concurrent.ThreadPoolExecutor` or `java.util.concurrent.ScheduledThreadPoolExecutor` will give you additional options.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Executor Interfaces
Next page: Fork/Join

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: High Level Concurrency Objects
Subsection: Executors

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Fork/Join

The fork/join framework is an implementation of the `ExecutorService` interface that helps you take advantage of multiple processors. It is designed for work that can be broken into smaller pieces recursively. The goal is to use all the available processing power to enhance the performance of your application.

As with any `ExecutorService` implementation, the fork/join framework distributes tasks to worker threads in a thread pool. The fork/join framework is distinct because it uses a *work-stealing* algorithm. Worker threads that run out of things to do can steal tasks from other threads that are still busy.

The center of the fork/join framework is the `ForkJoinPool` class, an extension of the `AbstractExecutorService` class. `ForkJoinPool` implements the core work-stealing algorithm and can execute `ForkJoinTask` processes.

Basic Use

The first step for using the fork/join framework is to write code that performs a segment of the work. Your code should look similar to the following pseudocode:

```
if (my portion of the work is small enough)
  do the work directly
else
  split my work into two pieces
  invoke the two pieces and wait for the results
```

Wrap this code in a `ForkJoinTask` subclass, typically using one of its more specialized types, either `RecursiveTask` (which can return a result) or `RecursiveAction`.

After your `ForkJoinTask` subclass is ready, create the object that represents all the work to be done and pass it to the `invoke()` method of a `ForkJoinPool` instance.

Blurring for Clarity

To help you understand how the fork/join framework works, consider the following example. Suppose that you want to blur an image. The original *source* image is represented by an array of integers, where each integer contains the color values for a single pixel. The blurred *destination* image is also represented by an integer array with the same size as the source.

Performing the blur is accomplished by working through the source array one pixel at a time. Each pixel is averaged with its surrounding pixels (the red, green, and blue components are averaged), and the result is placed in the destination array. Since an image is a large array, this process can take a long time. You can take advantage of concurrent processing on multiprocessor systems by implementing the algorithm using the fork/join framework. Here is one possible implementation:

```
public class ForkBlur extends RecursiveAction {
    private int[] mSource;
    private int mStart;
    private int mLength;
```

| Ad Choices

```
    private int[] mDestination;

    // Processing window size; should be odd.
    private int mBlurWidth = 15;

    public ForkBlur(int[] src, int start, int length, int[] dst) {
        mSource = src;
        mStart = start;
        mLength = length;
        mDestination = dst;
    }

    protected void computeDirectly() {
        int sidePixels = (mBlurWidth - 1) / 2;
        for (int index = mStart; index < mStart + mLength; index++) {
            // Calculate average.
            float rt = 0, gt = 0, bt = 0;
            for (int mi = -sidePixels; mi <= sidePixels; mi++) {
                int mindex = Math.min(Math.max(mi + index, 0),
                                    mSource.length - 1);
                int pixel = mSource[mindex];
                rt += (float)((pixel & 0x00ff0000) >> 16)
                      / mBlurWidth;
                gt += (float)((pixel & 0x0000ff00) >>  8)
                      / mBlurWidth;
                bt += (float)((pixel & 0x000000ff) >>  0)
                      / mBlurWidth;
            }

            // Reassemble destination pixel.
            int dpixel = (0xff000000     ) |
                   (((int)rt) << 16) |
                   (((int)gt) <<  8) |
                   (((int)bt) <<  0);
            mDestination[index] = dpixel;
        }
    }

  ...
```

Now you implement the abstract `compute()` method, which either performs the blur directly or splits it into two smaller tasks. A simple array length threshold helps determine whether the work is performed or split.

```
protected static int sThreshold = 100000;

protected void compute() {
    if (mLength < sThreshold) {
        computeDirectly();
        return;
    }

    int split = mLength / 2;

    invokeAll(new ForkBlur(mSource, mStart, split, mDestination),
              new ForkBlur(mSource, mStart + split, mLength - split,
                           mDestination));
}
```

If the previous methods are in a subclass of the `RecursiveAction` class, then setting up the task to run in a `ForkJoinPool` is straightforward, and involves the following steps:

1. Create a task that represents all of the work to be done.

```
// source image pixels are in src
// destination image pixels are in dst
ForkBlur fb = new ForkBlur(src, 0, src.length, dst);
```

| Ad Choices

```
ForkJoinPool pool = new ForkJoinPool();
```

3. Run the task.

```
pool.invoke(fb);
```

For the full source code, including some extra code that creates the destination image file, see the `ForkBlur` example.

Standard Implementations

Besides using the fork/join framework to implement custom algorithms for tasks to be performed concurrently on a multiprocessor system (such as the `ForkBlur.java` example in the previous section), there are some generally useful features in Java SE which are already implemented using the fork/join framework. One such implementation, introduced in Java SE 8, is used by the `java.util.Arrays` class for its `parallelSort()` methods. These methods are similar to `sort()`, but leverage concurrency via the fork/join framework. Parallel sorting of large arrays is faster than sequential sorting when run on multiprocessor systems. However, how exactly the fork/join framework is leveraged by these methods is outside the scope of the Java Tutorials. For this information, see the Java API documentation.

Another implementation of the fork/join framework is used by methods in the `java.util.streams` package, which is part of Project Lambda scheduled for the Java SE 8 release. For more information, see the Lambda Expressions section.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Thread Pools
Next page: Concurrent Collections

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: High Level Concurrency Objects

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Concurrent Collections

The `java.util.concurrent` package includes a number of additions to the Java Collections Framework. These are most easily categorized by the collection interfaces provided:

- `BlockingQueue` defines a first-in-first-out data structure that blocks or times out when you attempt to add to a full queue, or retrieve from an empty queue.
- `ConcurrentMap` is a subinterface of `java.util.Map` that defines useful atomic operations. These operations remove or replace a key-value pair only if the key is present, or add a key-value pair only if the key is absent. Making these operations atomic helps avoid synchronization. The standard general-purpose implementation of `ConcurrentMap` is `ConcurrentHashMap`, which is a concurrent analog of `HashMap`.
- `ConcurrentNavigableMap` is a subinterface of `ConcurrentMap` that supports approximate matches. The standard general-purpose implementation of `ConcurrentNavigableMap` is `ConcurrentSkipListMap`, which is a concurrent analog of `TreeMap`.

All of these collections help avoid Memory Consistency Errors by defining a happens-before relationship between an operation that adds an object to the collection with subsequent operations that access or remove that object.

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Fork/Join
Next page: Atomic Variables

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: High Level Concurrency Objects

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Atomic Variables

The `java.util.concurrent.atomic` package defines classes that support atomic operations on single variables. All classes have `get` and `set` methods that work like reads and writes on `volatile` variables. That is, a `set` has a happens-before relationship with any subsequent `get` on the same variable. The atomic `compareAndSet` method also has these memory consistency features, as do the simple atomic arithmetic methods that apply to integer atomic variables.

To see how this package might be used, let's return to the `Counter` class we originally used to demonstrate thread interference:

```
class Counter {
    private int c = 0;

    public void increment() {
        c++;
    }

    public void decrement() {
        c--;
    }

    public int value() {
        return c;
    }

}
```

One way to make `Counter` safe from thread interference is to make its methods synchronized, as in `SynchronizedCounter`:

```
class SynchronizedCounter {
    private int c = 0;

    public synchronized void increment() {
        c++;
    }

    public synchronized void decrement() {
        c--;
    }

    public synchronized int value() {
        return c;
    }
```

| Ad Choices

```
}
```

For this simple class, synchronization is an acceptable solution. But for a more complicated class, we might want to avoid the liveness impact of unnecessary synchronization. Replacing the `int` field with an `AtomicInteger` allows us to prevent thread interference without resorting to synchronization, as in `AtomicCounter`:

```
import java.util.concurrent.atomic.AtomicInteger;

class AtomicCounter {
    private AtomicInteger c = new AtomicInteger(0);

    public void increment() {
        c.incrementAndGet();
    }

    public void decrement() {
        c.decrementAndGet();
    }

    public int value() {
        return c.get();
    }

}
```

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Concurrent Collections
Next page: Concurrent Random Numbers

| Ad Choices

The Java™ Tutorials

Trail: Essential Java Classes
Lesson: Concurrency
Section: High Level Concurrency Objects

The Java Tutorials have been written for JDK 8. Examples and practices described in this page don't take advantage of improvements introduced in later releases and might use technology no longer available.
See Dev.java for updated tutorials taking advantage of the latest releases.
See Java Language Changes for a summary of updated language features in Java SE 9 and subsequent releases.
See JDK Release Notes for information about new features, enhancements, and removed or deprecated options for all JDK releases.

Concurrent Random Numbers

In JDK 7, `java.util.concurrent` includes a convenience class, `ThreadLocalRandom`, for applications that expect to use random numbers from multiple threads or `ForkJoinTask`s.

For concurrent access, using `ThreadLocalRandom` instead of `Math.random()` results in less contention and, ultimately, better performance.

All you need to do is call `ThreadLocalRandom.current()`, then call one of its methods to retrieve a random number. Here is one example:

```
int r = ThreadLocalRandom.current() .nextInt(4, 77);
```

About Oracle | Contact Us | Legal Notices | Terms of Use | Your Privacy Rights

Copyright © 1995, 2024 Oracle and/or its affiliates. All rights reserved.

Previous page: Atomic Variables
Next page: For Further Reading

| Ad Choices